markdown-validator Documentation

Contents

Chapter 1. Understanding markdown-validator....4

What is markdown-validator?....4

How validation works....5

Required and Suggested validation levels....7

How markdown-validator fits into your CI/CD pipeline....8

Working with multiple rule sets....10

Chapter 2. Get started....15

Set up markdown-validator from scratch....15

Install markdown-validator....18

Download NLTK language data....19

Chapter 3. Validate documents....21

Validate a Markdown file from the command line....21

Read and act on a validation report....22

Validate a directory of Markdown files....24

Set the output format for validation reports....25

Save validation reports to a directory....26

Chapter 4. CI/CD integration....29

Add markdown-validator to a GitHub Actions workflow....29

Chapter 5. Reference....33

CLI options reference....33

Exit codes reference....35

Validation report output reference....35

Chapter 6. Understanding rule sets....39

Understanding rule sets....39

Anatomy of a validation rule....41

Understanding the workflow step language....43

Rule design patterns for common editorial checks....47

Chapter 7. Author rules **52**

Create a rule set file 52

Write a header rule to check front-matter metadata 53

Write a body rule to check document structure 55

Write a workflow for conditional rule chains 57

Chapter 8. Test rules **59**

Start the interactive REPL 59

Test a rule in the interactive REPL 60

Test a workflow in the interactive REPL 61

Chapter 9. Govern and deploy **63**

Rule set governance and versioning 63

Known limitations 64

Deploy and version a rule set 66

Chapter 10. Reference **68**

Rule fields reference 68

Flags reference 69

Operators reference 71

Workflow step patterns reference 72

REPL commands reference 74

Chapter 11. Troubleshooting **76**

Resolve a NLTK LookupError on first run 76

Resolve a ParseError caused by missing front matter 77

Debug a rule that never passes on a valid document 78

Debug an XPath expression that returns unexpected results 80

Resolve a validation failure in a CI/CD pipeline 82

Chapter 12. Reference **84**

Glossary 84

Chapter 1. Understanding markdown-validator

What is markdown-validator?

markdown-validator is a rule-based tool that checks your Markdown files against a set of editorial standards defined by your team. It catches missing metadata, incorrect front-matter values, and structural problems before your content reaches review or publication.

> **Note:**
> **Applies to:** markdown-validator 0.2.x. This documentation was last updated 2026-03-20. Planned changes in v0.3 (negation operator) and v0.4 (structured DOM) may affect rule behaviour described here.

> **Important:**
> **If your pull request was just blocked by a validation failure:** Go directly to Validate a Markdown file from the command line *(on page 21)* to reproduce the failure locally, then Read and act on a validation report *(on page 22)* to identify and fix the specific issues. You do not need to read this entire topic first.

The problem it solves

When a documentation repository contains hundreds or thousands of articles contributed by many authors, maintaining consistent quality by hand is not practical. Reviewers spend time flagging the same issues repeatedly — a missing `description` field, a stale `ms.date`, an article that uses the wrong template heading structure. These are mechanical checks that a tool can perform automatically.

markdown-validator runs those checks for you. Your team defines the rules once in a JSON file; the tool applies them to every article in the repository and reports exactly which rules each file passes or fails.

What it checks

markdown-validator can check two parts of every Markdown file:

Front-matter metadata

The YAML block at the top of the file, delimited by `---`. Common checks include: required fields are present, field values match allowed values, the date field is recent, and the title follows a naming convention.

Document body

The article content below the front matter. Common checks include: the document has exactly one H1 heading, required sections are present (such as "Prerequisites" and "Next steps"), headings follow the right capitalization pattern, and no sections are empty.

How authors use it

As a content author, you use markdown-validator in two ways:

1. **Locally before opening a pull request.** Run the tool against your article from the command line to catch problems before your reviewers see them. If a rule fails, the output tells you exactly what to fix.
2. **Automatically in CI/CD.** Your team's CI pipeline runs the same validation on every pull request. If any required rule fails, the pipeline blocks the merge and shows you the report.

You do not need to understand how rules are written to use this tool. Your content lead maintains the rule set; you run the validator and fix what it reports.

What you need to get started

- Python 3.12 or later installed on your computer
- The rule set file for your documentation project (ask your content lead for the path to this file)
- A Markdown file to validate

Next steps

- To understand how validation works, see How validation works *(on page 5)*.
- To install the tool, see Install markdown-validator *(on page 18)*.

How validation works

markdown-validator processes your file in three stages: it parses the front matter and document body, evaluates each rule in the rule set against your file, and produces a scored report that identifies every rule your file passes or fails.

The three stages

Figure 1. The three-stage validation pipeline

Source: `docs-user/images/validation-pipeline.mmd`

Stage 1: Parse

The tool reads your `.md` file and splits it into two parts:

- **Front matter.** The YAML block delimited by `---` at the top of the file. The tool extracts all key-value pairs from this block.
- **Document body.** The Markdown content below the front matter. The tool renders this to HTML so it can be queried structurally.

> **Important:**
> Your Markdown file must begin with a front-matter block. Files without a `---` opening block produce a parse error and cannot be validated. See Resolve a ParseError caused by missing front matter *(on page 77)* if this happens to you.

Stage 2: Evaluate

The tool applies each rule in the rule set to your file, one at a time. Each rule either passes or fails. Rules are independent; a failing rule does not stop evaluation of the remaining rules.

Some rules are grouped into *workflows* — sequences that apply conditional logic. For example, a workflow might check heading structure only when a specific metadata value is present. As an author, you do not need to understand how workflows are written; the report shows you their outcome just as it shows individual rule results.

Stage 3: Report

The tool produces a report that shows:

- A **score** — the number of rules that passed out of the total number evaluated (for example, `24/26`)
- A **pass/fail status** for each rule
- A **mitigation message** for each failed rule, explaining what to fix

What the score means

Your score is the count of rules that passed divided by the total number of rules in the rule set. A score of `26/26` means your file passes every check. A score of `24/26` means two rules failed.

Not all failed rules block your pull request. Your team classifies each rule as either **Required** or **Suggested**:

- A **Required** rule failure blocks the CI pipeline. You must fix it before your pull request can merge.
- A **Suggested** rule failure appears in the report but does not block the merge. It is a recommendation, not a hard requirement.

For more about these levels, see Required and Suggested validation levels *(on page 7)*.

Example validation output

Here is what a typical text-format report looks like when two rules fail:

```
Validating: docs/tutorial-deploy-app.md

Rule set:   rules/tutorial.json


  ✓  [1]  topic-must-be-tutorial

  ✓  [2]  must-have-h1

  ✓  [3]  h1-must-start-with-tutorial

  ✗  [4]  description-must-exist

          → Add a description field to the front matter.

  ✓  [5]  ms-date-must-be-fresh

  ✗  [6]  title-under-60-chars

          → Title must be under 60 characters.


Score: 4/6   FAILED
```

The check mark (✓) means the rule passed. The cross (✗) means it failed. The arrow (→) shows the mitigation message — exactly what you need to fix.

Required and Suggested validation levels

Every rule in a rule set has a level: **Required** or **Suggested**. Required failures block your CI pipeline. Suggested failures appear in the report as recommendations and do not block a merge.

What the levels mean

Table 1. Validation level comparison

Level	Effect on the pipeline	When your team sets this level
Required	CLI exits with code `1`. CI pipeline fails and blocks the merge.	The check enforces a hard editorial standard — for example, every tutorial must

Table 1. Validation level comparison (continued)

Level	Effect on the pipeline	When your team sets this level
		have an `ms.topic: tutorial` value and a single H1.
Suggested	CLI exits with code `0` even if the rule fails. The failure appears in the report but does not block the merge.	The check is a best-practice recommendation — for example, title length should be under 60 characters. Violations are noted but not enforced.

What you see as an author

The report labels each rule with its level so you can distinguish hard failures from recommendations at a glance:

```
✗  [4]  description-must-exist          [Required]

         → Add a description field to the front matter.


✗  [6]  title-under-60-chars            [Suggested]

         → Title must be under 60 characters.
```

In this example, rule 4 is a **Required** failure that will block your pull request. Rule 6 is a **Suggested** failure that is worth addressing but will not prevent the merge.

The default level

If a rule in the rule set does not specify a level, it defaults to **Required**. Ask your content lead if you are unsure whether a specific rule is Required or Suggested.

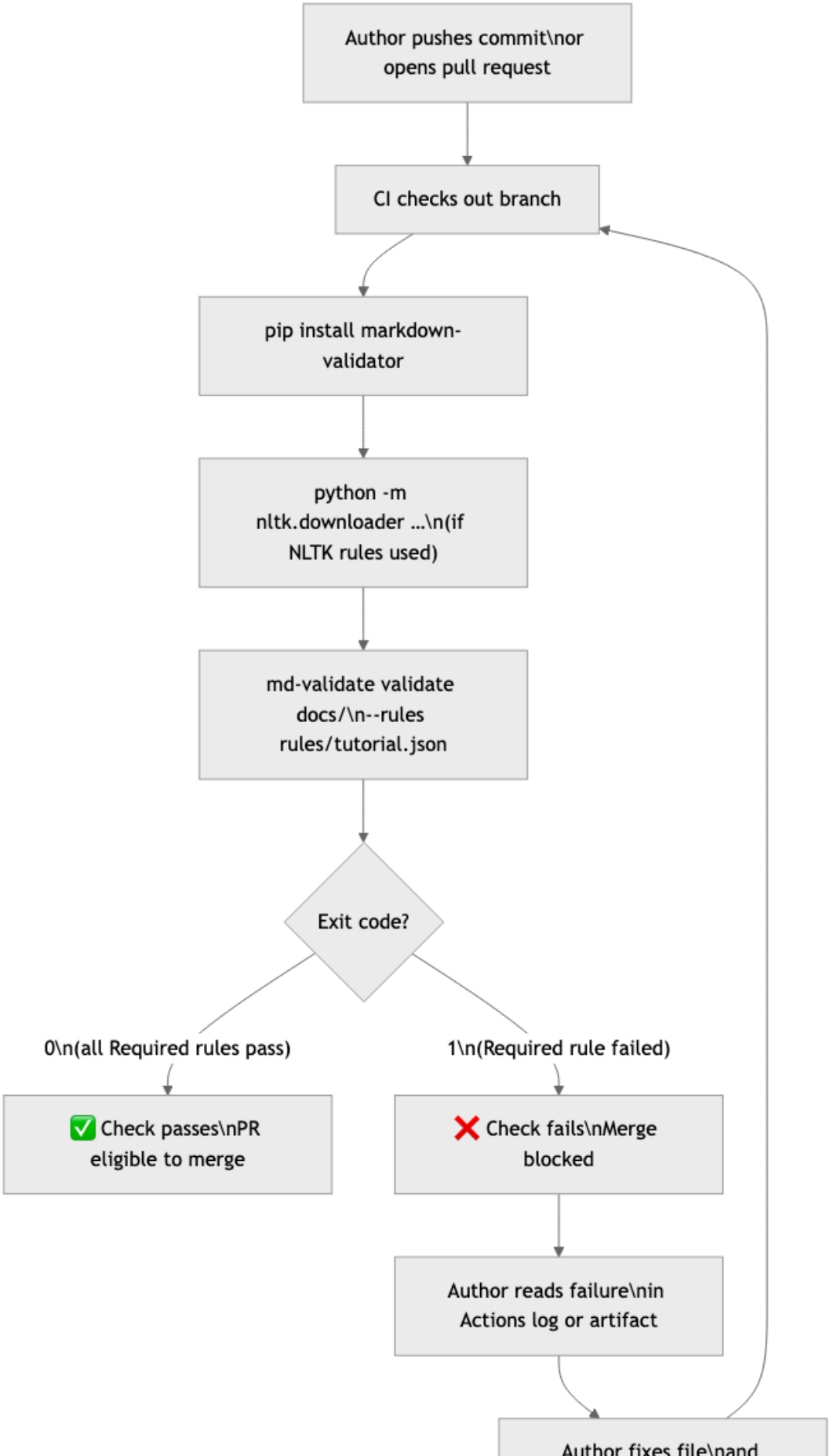
How markdown-validator fits into your CI/CD pipeline

When markdown-validator runs in your CI/CD pipeline, it acts as a quality gate on every pull request. If any Required rule fails, the pipeline exits with a non-zero code and blocks the merge until you fix the reported issues.

The CI validation workflow

This diagram shows where validation fits in a typical pull request workflow:

Figure 2. Pull-request validation flow



Source: `docs-user/images/ci-integration-flow.mmd`

1. You push a commit or open a pull request against the documentation repository.
2. The CI pipeline checks out your branch and runs `md-validate validate` against the changed files using your team's rule set.
3. If all Required rules pass, the pipeline exits with code `0` and the pull request is eligible to merge.
4. If any Required rule fails, the pipeline exits with code `1`. The merge is blocked. The pipeline log shows which rules failed and what you need to fix.
5. You fix your file, push the commit, and the pipeline re-runs.

What the pipeline runs

Your team's pipeline runs a command similar to this:

```
md-validate validate docs/ --rules rules/tutorial.json
```

The exact path to the rule set file and the target directory vary by project. Ask your content lead or check your repository's CI configuration file (typically `.github/workflows/validate-docs.yml` for GitHub Actions) to find the exact command your project uses.

Local validation vs. CI validation

Running validation locally before opening a pull request saves time. You catch and fix problems before they block the pipeline, avoiding the round-trip of a failed CI run.

The rule set used locally and in CI should be the same file, stored in the repository. If your local run produces different results from CI, check that you are using the correct rule set file and the same version of markdown-validator.

Set up CI integration

To add markdown-validator to a GitHub Actions workflow, see Add markdown-validator to a GitHub Actions workflow *(on page 29)*.

Working with multiple rule sets

A single repository can contain content that must meet different editorial standards — for example, a tutorial section and an API reference section with distinct metadata requirements. Use separate rule set files and specify the correct one at runtime with the `--rules` option.

When to use more than one rule set

A single rule set works well when all content in the repository follows the same standards. Use multiple rule sets when:

- **Different content types have different metadata requirements.** For example, tutorial articles must have `ms.topic: tutorial` and a "Prerequisites" section, while API reference articles do not.
- **Different teams own different subdirectories** and maintain their own editorial standards independently.
- **You are migrating from one standard to another** and need to run the old rule set against existing content and the new rule set against new content simultaneously.
- **One rule set is shared across repositories** (for example, a base style standard) while each repository adds its own product-specific rules on top.

Specifying the rule set at runtime

The `--rules` option accepts any path to a JSON rule set file. You can run the validator multiple times against different directories using different rule sets:

```
# Validate tutorials with the tutorial rule set

md-validate validate docs/tutorials/ --rules rules/tutorials.json



# Validate API reference with the reference rule set

md-validate validate docs/reference/ --rules rules/api-reference.json
```

There is no configuration file that automatically selects the rule set. The rule set is always specified explicitly on the command line — either by the author running the tool locally or by the CI pipeline.

How authors know which rule set applies

Authors need to know which rule set to use before they can validate their files locally. Establish a clear convention and document it in your repository's contributing guide. Common approaches:

- **Convention by directory:** a specific rule set always applies to a specific directory. For example, all files under `docs/tutorials/` are validated with `rules/tutorials.json`. Authors look up the directory-to-ruleset mapping in the contributing guide.
- **Convention by front-matter field:** authors set `ms.topic` in their front matter and the contributing guide maps each topic type to a rule set.
- **Script wrapper:** the repository provides a `validate.sh` script that selects the rule set automatically based on the file path passed to it. Authors run the script instead of calling `md-validate` directly.

Whichever approach you use, the rule set path that authors use locally must match the one used in CI. See Preventing local vs CI rule mismatch *(on page 13)* below.

CI configuration for multiple rule sets

When you have two or more rule sets, each one should run as a separate CI job so that failures in one area do not suppress failures in another. Use a GitHub Actions matrix strategy to avoid duplicating the job definition:

```
jobs:
  validate:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        include:
          - docs-dir: docs/tutorials/
            rules-file: rules/tutorials.json
            artifact-name: reports-tutorials
          - docs-dir: docs/reference/
            rules-file: rules/api-reference.json
            artifact-name: reports-reference

    steps:
      - name: Check out repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'
```

```
    - name: Cache NLTK data
      uses: actions/cache@v4
      with:
        path: ~/nltk_data
        key: nltk-punkt-tagger

    - name: Install markdown-validator
      run: pip install markdown-validator

    - name: Download NLTK language data
      run: python -m nltk.downloader punkt_tab averaged_perceptron_tagger_eng

    - name: Validate documentation
      run: |
        md-validate validate ${{ matrix.docs-dir }} \
          --rules ${{ matrix.rules-file }} \
          --format json \
          --output reports/

    - name: Upload validation reports
      if: always()
      uses: actions/upload-artifact@v4
      with:
        name: ${{ matrix.artifact-name }}
        path: reports/
```

Each matrix entry produces a separate check on the pull request. A failure in tutorials does not prevent the reference check from running and reporting its own result.

Preventing local vs CI rule mismatch

The most common multi-ruleset problem is an author validating with the wrong rule set locally, then seeing different failures in CI. To prevent this:

- **Commit all rule set files to the repository.** Do not let individual team members maintain local copies that diverge from the version in source control. The CI pipeline reads rule sets from the committed files, so any local deviation will cause results to differ.
- **Document the exact command to run locally** in the contributing guide, including the full `--rules` path relative to the repository root. Avoid instructions like "use the rules file" without specifying which one.
- **Use the same rule set path in CI as in the docs.** Check the `--rules` value in the workflow file and confirm it matches the path documented for local use.
- **Pin the rule set to a known version** if rule sets are shared across repositories via a package or git submodule. An uncoordinated update to a shared rule set can cause CI failures that are difficult to trace.

If local validation passes but CI fails — or vice versa — the first thing to check is whether the `--rules` path is identical in both contexts. See also Resolve a validation failure in a CI/CD pipeline *(on page 82)* for a step-by-step diagnosis.

Chapter 2. Get started

Set up markdown-validator from scratch

This scenario walks you through the complete setup of markdown-validator — from installation to a running CI gate — in a single guided sequence. Follow it top to bottom the first time you set up the tool for a documentation project.

Before you begin, confirm you have the following:

- Python 3.12 or later installed on your computer. To check:

  ```
  python --version
  ```

- A documentation repository with at least one `.md` file to validate.
- The path to your team's rule set file. If you do not have one, ask your content lead. The file is typically at `rules/tutorial.json` in your repository.
- Write access to the repository's `.github/workflows/` directory (for the CI step).

This walkthrough covers five stages:

1. **Install** — Install markdown-validator in a virtual environment.
2. **Download language data** — Install the NLTK corpora needed for sentence and part-of-speech rules.
3. **Validate a file** — Run your first validation and read the report.
4. **Fix failures** — Interpret the report and resolve Required failures.
5. **Add CI** — Configure a GitHub Actions workflow to validate on every pull request.

Each stage links to a detailed task if you need more guidance on any individual step.

1. Create and activate a Python virtual environment.

   On macOS or Linux:

   ```
   python -m venv .venv

   source .venv/bin/activate
   ```

   On Windows (PowerShell):

   ```
   python -m venv .venv

   .venv\Scripts\Activate.ps1
   ```

   Your prompt changes to show `(.venv)` when the environment is active.

2. Install markdown-validator.

```
pip install markdown-validator
```

pip installs markdown-validator and its dependencies. Verify with:

```
md-validate --version
```

3. Download the NLTK language data required by sentence-count and part-of-speech rules.

```
python -m nltk.downloader punkt_tab averaged_perceptron_tagger_eng
```

> **Tip:**
> If your team's rule set does not use sentence-count (`s`) or part-of-speech (`p<N>`) rules, you can skip this step. Ask your content lead if you are unsure. For full details, see Download NLTK language data *(on page 19)*.

NLTK downloads the corpora to your home directory (`~/nltk_data` on macOS/Linux). You only need to do this once per Python environment.

4. Run your first validation against a Markdown file.

```
md-validate validate docs/my-article.md --rules rules/tutorial.json
```

Replace `docs/my-article.md` with the path to one of your files, and `rules/tutorial.json` with the path your content lead provided.

The tool prints a validation report. A fully passing report ends with:

```
Score: 8/8   PASSED
```

A report with failures ends with:

```
Score: 6/8   FAILED
```

5. Read the report and identify Required failures.

Each failed rule shows a cross (✗), the rule name, and a mitigation message:

```
✗  [4]  description-must-exist          [Required]

        → Add a description field to the front matter.
```

Required failures will block your pull request. Suggested failures are recommendations — they do not block the merge. For a full guide to reading the report, see Read and act on a validation report *(on page 22)*.

6. Open your Markdown file and fix each Required failure using its mitigation message.

Common front-matter fixes:

- **Missing field** — add the key and value to the `---` block at the top of the file
- **Wrong value** — change the field value to match what the mitigation message specifies
- **Stale date** — update `ms.date` to today in `YYYY-MM-DD` format

Common body fixes:

- **Missing H1** — ensure exactly one line begins with a single `#`
- **H1 format** — edit the H1 to match the prefix in the mitigation message

7. Re-run validation to confirm all Required failures are resolved.

```
md-validate validate docs/my-article.md --rules rules/tutorial.json
```

The report shows PASSED when all Required rules pass.

8. Create the GitHub Actions workflow directory if it does not exist.

```
mkdir -p .github/workflows
```

9. Create `.github/workflows/validate-docs.yml` using the complete workflow template in Add markdown-validator to a GitHub Actions workflow *(on page 29)*.

The workflow template in that topic is the authoritative version. Copy it verbatim, then update the two values shown below to match your repository:

- `paths: 'docs/**/*.md'` — the glob pattern for your documentation files
- `--rules rules/tutorial.json` — the path to your team's rule set file

The workflow installs markdown-validator, caches and downloads NLTK data, validates the docs directory, and uploads per-file JSON reports as a downloadable artifact — even when validation fails.

10. Commit the workflow file and push.

```
git add .github/workflows/validate-docs.yml

git commit -m "Add documentation validation workflow"

git push
```

GitHub runs the workflow on every subsequent pull request that changes `.md` files matching the path filter. Required failures block the merge.

11. Open a test pull request to verify the workflow runs correctly.

Navigate to the **Actions** tab of your repository and confirm the **Validate documentation** check appears and completes. If it fails on a file that passed locally, check that CI is using the same rule set path you tested with.

markdown-validator is installed, your documents validate locally, and your GitHub Actions workflow blocks pull requests that fail Required rules. Every author on the project can now run `md-validate validate` locally to catch issues before pushing.

Now that validation is running, share the following with your team:

- The command to install and run validation locally (steps 1–4 of this topic)
- The path to the rule set file
- Where to find mitigation guidance: Read and act on a validation report *(on page 22)*
- Who to contact when a rule seems wrong: your content lead

To validate the entire repository before a release, see Validate a directory of Markdown files *(on page 24)*.

Install markdown-validator

Install markdown-validator with pip in a Python virtual environment. After installation, the `md-validate` command is available from your terminal.

- Python 3.12 or later must be installed. To check your Python version, open a terminal and enter:

  ```
  python --version
  ```

  If Python is not installed or is older than 3.12, download it from python.org.
- You need access to a terminal (Command Prompt or PowerShell on Windows, Terminal on macOS or Linux).

Installing markdown-validator inside a virtual environment keeps its dependencies isolated from other Python projects on your computer. This is the recommended approach for any Python tool you use regularly.

1. Create and activate a virtual environment.

   On macOS or Linux:

   ```
   python -m venv .venv

   source .venv/bin/activate
   ```

   On Windows (PowerShell):

   ```
   python -m venv .venv

   .venv\Scripts\Activate.ps1
   ```

When the virtual environment is active, your terminal prompt includes `(.venv)`.

The virtual environment is active. Packages you install now go into this isolated environment.

2. Install markdown-validator.

```
pip install markdown-validator
```

pip downloads and installs markdown-validator and its dependencies.

3. Verify the installation.

```
md-validate --version
```

The terminal displays the installed version number, confirming that the tool is ready to use.

markdown-validator is installed. You can now run `md-validate validate` from this terminal session whenever the virtual environment is active.

Some validation rules check sentence counts and word-level grammar using the NLTK language library. These rules require additional language data that is not included in the pip package. If your team's rule set uses these checks, complete the next step: Download NLTK language data *(on page 19)*.

To run your first validation, see Validate a Markdown file from the command line *(on page 21)*.

Download NLTK language data

Download the NLTK language corpora required by validation rules that check sentence counts or part-of-speech tags. You only need to do this once per Python environment.

markdown-validator must be installed. See Install markdown-validator *(on page 18)*.

Some validation rules check properties of natural language — for example, "the introduction must be no more than three sentences" or "the first word of the H1 must be a verb." These rules use the NLTK (Natural Language Toolkit) library, which requires separate language data files called *corpora*.

If these files are not present on your system, any rule that uses them fails with a `LookupError: Resource punkt_tab not found` message. Downloading the corpora once resolves this permanently for the current Python environment.

> **Tip:**
> Ask your content lead whether your team's rule set uses sentence-count (`s`) or part-of-speech (`p<N>`) rules. If it does not, you can skip this step.

1. Activate the virtual environment where markdown-validator is installed.

   On macOS or Linux:

   ```
   source .venv/bin/activate
   ```

   On Windows (PowerShell):

   ```
   .venv\Scripts\Activate.ps1
   ```

2. Download the required NLTK corpora.

   ```
   python -m nltk.downloader punkt_tab averaged_perceptron_tagger_eng
   ```

   This command downloads two datasets:

   - `punkt_tab` — sentence tokenizer used for sentence-count rules
   - `averaged_perceptron_tagger_eng` — part-of-speech tagger used for POS rules

   NLTK downloads the files to your NLTK data directory (typically `~/nltk_data` on macOS and Linux, or `%APPDATA%\nltk_data` on Windows).

3. Verify the download succeeded.

   ```
   python -c "import nltk; nltk.data.find('tokenizers/punkt_tab'); print('OK')"
   ```

   The terminal prints `OK`. If it prints a LookupError instead, the download did not complete. Run the download command again.

The NLTK language data is installed. Sentence-count and part-of-speech rules will now evaluate correctly.

You are ready to validate files. See Validate a Markdown file from the command line *(on page 21)*.

If you see a LookupError during validation even after this step, see Resolve a NLTK LookupError on first run *(on page 76)*.

Chapter 3. Validate documents

Validate a Markdown file from the command line

Run `md-validate validate` against a single Markdown file to check it against your team's rule set. The output shows which rules passed, which failed, and what you need to change.

- markdown-validator is installed and the virtual environment is active. See Install markdown-validator *(on page 18)*.
- You have the path to your team's rule set file. If you are unsure, ask your content lead. The rule set is typically stored in the documentation repository under a folder named `rules/`.

Validating your article locally before opening a pull request lets you catch and fix problems before the CI pipeline sees them. Fix all Required failures before you push; you can choose whether to address Suggested failures.

1. Open a terminal and navigate to your documentation repository.

   ```
   cd path/to/your/docs-repo
   ```

2. Activate the virtual environment.

   On macOS or Linux:

   ```
   source .venv/bin/activate
   ```

   On Windows (PowerShell):

   ```
   .venv\Scripts\Activate.ps1
   ```

3. Run the validate command with the path to your file and your team's rule set.

   ```
   md-validate validate docs/my-article.md --rules rules/tutorial.json
   ```

   Replace `docs/my-article.md` with the path to your file and `rules/tutorial.json` with the path to your team's rule set.

   The tool prints a validation report to the terminal.

4. Review the output and identify any failed rules.

   A passing report looks like this:

   ```
   ✓  [1]  topic-must-be-tutorial
   ✓  [2]  must-have-h1
   ✓  [3]  description-must-exist
   ```

```
Score: 3/3   PASSED
```

A report with failures looks like this:

```
  ✓  [1]  topic-must-be-tutorial

  ✗  [2]  must-have-h1

          → The document must have exactly one H1 heading.

  ✓  [3]  description-must-exist


Score: 2/3   FAILED
```

Rules marked with `✗` failed. The arrow (`→`) shows the mitigation message — a plain-English description of what to fix.

5. **Optional:** Fix the failures identified in the report.

   Open your Markdown file in your editor and address each failing rule using the mitigation message as your guide. For detailed guidance on interpreting and acting on a report, see Read and act on a validation report *(on page 22)*.

6. **Optional:** Re-run the validate command to confirm all failures are resolved.

   ```
   md-validate validate docs/my-article.md --rules rules/tutorial.json
   ```

   The score and PASSED/FAILED status reflect your changes.

Your file either passes all Required rules (score shows PASSED) or shows the remaining failures to fix.

- To understand what each part of the report means and how to fix specific failure types, see Read and act on a validation report *(on page 22)*.
- To validate an entire folder of files at once, see Validate a directory of Markdown files *(on page 24)*.

Read and act on a validation report

A validation report shows which rules your file passes or fails, the severity of each failure, and a plain-English mitigation message that tells you exactly what to fix. This topic walks you through reading the report and taking action.

After running `md-validate validate`, you see a report in the terminal. Not all failures are equal: Required failures block your pull request; Suggested failures are recommendations. Prioritize Required failures first.

1. Look at the score line at the bottom of the report.

```
Score: 4/6   FAILED
```

   The score shows how many rules passed (4) out of the total (6). `FAILED` means at least one Required rule failed. `PASSED` means all Required rules passed (even if some Suggested rules failed).

2. Find every line that starts with `✗`.

   These lines identify the rules your file failed. Each failure shows the rule ID in square brackets, the rule name, and a mitigation message on the next line:

```
✗  [4]  description-must-exist

         → Add a description field to the front matter.
```

3. Identify which failures are Required (blocking).

   Required failures show `[Required]` in the report. If the report does not show a level label, the rule defaults to Required. Fix all Required failures before pushing your branch.

4. Fix front-matter failures by editing the YAML block at the top of your file.

   The most common front-matter failures and their fixes:

   Table 2. Common front-matter failures

Rule type	What failed	How to fix it
Missing field	A required key (such as `description`) is absent	Add the field to the YAML block: `description: "Brief description here."`
Wrong value	`ms.topic` is set to an unexpected value	Change the value to match the expected one shown in the mitigation message
Stale date	`ms.date` is too old	Update `ms.date` to today's date in `YYYY-MM-DD` format
Title format	Title does not start with the required prefix	Edit the `title` field to match the pattern in the mitigation message

5. Fix body failures by editing the Markdown content below the front matter.

   The most common body failures and their fixes:

Table 3. Common body failures

Rule type	What failed	How to fix it
Missing H1	The document has no H1 or more than one H1	Ensure exactly one line begins with a single `#`
H1 format	H1 does not start with the required prefix (such as "Tutorial:")	Edit the H1 to match the pattern in the mitigation message
Missing section	A required H2 (such as "Prerequisites") is absent	Add the required H2 section and content

6. Re-run validation to confirm all Required failures are resolved.

```
md-validate validate docs/my-article.md --rules rules/tutorial.json
```

The report shows PASSED when all Required rules pass.

Your file passes all Required rules and is ready for review.

Validate a directory of Markdown files

Run `md-validate validate` with a directory path to validate all Markdown files in that directory and its subdirectories at once. Use this before a release or periodic quality review.

markdown-validator is installed and the virtual environment is active. See Install markdown-validator *(on page 18)*.

When you pass a directory path instead of a file path, markdown-validator scans every `.md` file in that directory recursively. It produces one report per file and a summary in the terminal.

A single failing file sets the exit code to `1` for the entire run. This is the behavior CI pipelines rely on to gate the merge.

1. Run the validate command with a directory path.

```
md-validate validate docs/ --rules rules/tutorial.json
```

Replace `docs/` with the path to the directory you want to scan and `rules/tutorial.json` with your team's rule set.

The tool prints a report for each file it validates, followed by a summary of how many files passed and failed.

2. Review the summary at the end of the output.

```
Summary: 47 files validated — 45 passed, 2 failed
```

   Scroll up to find the per-file reports for the two failed files. The file path is shown at the top of each report.

3. **Optional:** To save per-file reports, add the `--output` option.

```
md-validate validate docs/ --rules rules/tutorial.json --output reports/
```

   The tool creates one report file per validated Markdown file in the `reports/` directory. This is useful for sharing results with your team or tracking quality over time. For more about saving reports, see Save validation reports to a directory *(on page 26)*.

Every Markdown file in the directory has been validated. Files with Required failures are identified in the terminal output and, if you used `--output`, in the saved report files.

Set the output format for validation reports

Use the `--format` option to choose how markdown-validator presents its report: plain text for terminal reading, JSON for CI integration and scripting, or CSV for spreadsheet analysis.

The default format is `text`, which is readable in the terminal. Use `json` when you need to parse the output in a script or CI pipeline, and `csv` when you want to share results in a spreadsheet or import them into a reporting tool.

1. To produce human-readable text output (the default), run the validate command without the `--format` option, or specify `--format text`.

```
md-validate validate docs/article.md --rules rules/tutorial.json
```

   Example output:

```
  ✓  [1]  topic-must-be-tutorial

  ✗  [4]  description-must-exist

          → Add a description field to the front matter.


Score: 5/6   FAILED
```

2. To produce JSON output for scripting or CI integration, add `--format json`.

```
md-validate validate docs/article.md --rules rules/tutorial.json --format json
```

   The JSON output includes all rule results and metadata:

```
{
  "filepath": "docs/article.md",
  "score": 5,
  "total_rules": 6,
  "passed": false,
  "results": [
    {"rule_id": 1, "rule_name": "topic-must-be-tutorial", "passed": true, ...},
    {"rule_id": 4, "rule_name": "description-must-exist",  "passed": false,
     "mitigation": "Add a description field to the front matter.", ...}
  ]
}
```

3. To produce CSV output for spreadsheet analysis, add `--format csv`.

```
md-validate validate docs/ --rules rules/tutorial.json --format csv --output reports/
```

Each row in the CSV file represents one rule evaluation. Columns include the rule ID, name, type, level, and pass/fail status.

> **Tip:**
> CSV output is most useful when combined with `--output` to save the file. See Save validation reports to a directory *(on page 26)*.

The report is produced in the format you specified.

Save validation reports to a directory

Use the `--output` option to write one report file per validated document to a directory. This is useful for sharing results with your team, uploading reports as CI artifacts, or tracking quality over time.

Decide on a format for the saved reports before running the command. See Set the output format for validation reports *(on page 25)* for a comparison of text, JSON, and CSV formats.

Saving reports to a directory is useful in three situations:

CI artifact upload

The CI pipeline saves JSON reports to a directory, then uploads the directory as a build artifact. Authors can download the artifact from the Actions run summary page to read the full validation output without scrolling through CI logs.

Quality tracking over time

Save CSV reports to a versioned directory after each run. The CSV format is spreadsheet-friendly and easy to import into a quality dashboard or compare across runs using standard diff tools.

Team review

Save text reports and share the directory with reviewers. Text format is readable without tooling and works well for ad-hoc review sessions or email attachments.

The output directory does not need to exist before you run the command — markdown-validator creates it automatically.

1. Choose the report format that matches your use case.

   JSON — for CI artifact upload and machine processing

   ```
   md-validate validate docs/ --rules rules/tutorial.json \
     --format json --output reports/
   ```

   CSV — for quality tracking and spreadsheet import

   ```
   md-validate validate docs/ --rules rules/tutorial.json \
     --format csv --output reports/
   ```

   Text — for human review and sharing

   ```
   md-validate validate docs/ --rules rules/tutorial.json \
     --format text --output reports/
   ```

   Replace `reports/` with the path to the directory where you want the report files saved.

   The tool validates every file and writes one report file per document into the output directory. The file extension matches the format: `.json`, `.csv`, or `.txt`.

2. Open the output directory to confirm the generated files.

   Each report file is named after the source document. For example, validating `docs/tutorial-deploy-app.md` with `--format json` produces `reports/tutorial-deploy-app.json`. With `--format csv` it produces `reports/tutorial-deploy-app.csv`.

   If you validate a directory, the output directory mirrors only the filenames — subdirectory paths from the source are not recreated. All report files land in the single output directory you specified.

3. Adopt a naming convention for the output directory if you save reports from multiple runs.

   Including a date or build identifier in the directory name makes it easy to compare reports across runs:

```
md-validate validate docs/ --rules rules/tutorial.json \
  --format csv --output reports/2026-03-20/
```

For CI pipelines, use the run number or commit SHA to make each set of reports uniquely identifiable.

One report file per validated document is saved in the output directory in the format you specified. The exit code is `0` if all Required rules pass, or `1` if any Required rule fails — regardless of whether reports are saved.

To upload report files as CI artifacts (so your team can download them from the Actions run summary page), see Add markdown-validator to a GitHub Actions workflow *(on page 29)*.

Chapter 4. CI/CD integration

Add markdown-validator to a GitHub Actions workflow

Create a GitHub Actions workflow file that installs markdown-validator, downloads NLTK language data, validates your documentation on every pull request, and uploads reports as downloadable artifacts.

- Your documentation repository is hosted on GitHub.
- The rule set file is committed to the repository.
- You have write access to the repository to create workflow files.

A GitHub Actions workflow runs automatically when you open or update a pull request. If the validation step exits with code `1` (one or more Required rules failed), GitHub marks the check as failed and prevents the merge. Authors can see the report in the Actions log or as a downloaded artifact.

1. In your repository, create the directory `.github/workflows/` if it does not exist.

```
mkdir -p .github/workflows
```

2. Create a new file named `.github/workflows/validate-docs.yml` and paste in the following workflow. Do not modify it yet — update the two project-specific values in the next step.

```
name: Validate documentation


on:
  pull_request:
    paths:
      - 'docs/**/*.md'


jobs:
  validate:
    runs-on: ubuntu-latest


    steps:
      - name: Check out repository
        uses: actions/checkout@v4


      - name: Set up Python
        uses: actions/setup-python@v5
        with:
```

```
        python-version: '3.12'

    - name: Cache NLTK data
      uses: actions/cache@v4
      with:
        path: ~/nltk_data
        key: nltk-punkt-tagger

    - name: Install markdown-validator
      run: pip install markdown-validator

    - name: Download NLTK language data
      run: |
        python -m nltk.downloader punkt_tab averaged_perceptron_tagger_eng

    - name: Validate documentation
      run: |
        md-validate validate docs/ \
          --rules rules/tutorial.json \
          --format json \
          --output reports/

    - name: Upload validation reports
      if: always()
      uses: actions/upload-artifact@v4
      with:
        name: validation-reports
        path: reports/
```

3. Review what each workflow step does before customizing.

Check out repository

Checks out your repository so subsequent steps can read the files.

Set up Python

Installs Python 3.12, which markdown-validator requires.

Cache NLTK data

Saves the downloaded NLTK corpora between runs so the pipeline does not re-download them on every pull request.

Install markdown-validator

Installs the tool from PyPI.

Download NLTK language data

Downloads the `punkt_tab` and `averaged_perceptron_tagger_eng` corpora that sentence-count and part-of-speech rules require. Runs even when the cache is warm; the downloader skips files already present.

Validate documentation

Runs markdown-validator across your docs directory and writes one JSON report per file into `reports/`. The step exits with code `1` if any Required rule fails, which marks the GitHub check as failed and blocks the merge.

Upload validation reports

Uploads the `reports/` directory as a downloadable artifact named **validation-reports**. The `if: always()` condition ensures the upload runs even when the validation step fails, so authors can download the report to see what to fix.

4. Update the workflow to match your repository's paths and rule set location.

   Change the following values:

   - `docs/**/*.md` — the glob pattern matching your documentation files
   - `docs/` (in the validate command) — the directory to scan
   - `rules/tutorial.json` — the path to your team's rule set file

5. Commit the workflow file and push the branch.

```
git add .github/workflows/validate-docs.yml

git commit -m "Add documentation validation workflow"

git push
```

   GitHub runs the workflow on every subsequent pull request that changes files matching the path filter.

6. Open a pull request to verify the workflow runs correctly.

   Navigate to the **Actions** tab of your repository and select the latest workflow run to see the validation output. If the workflow fails, the output shows which rules failed and what to fix.

Every pull request that changes Markdown files in your repository now triggers a validation run. Required failures block the merge and notify the author.

Chapter 5. Reference

CLI options reference

Complete reference for all `md-validate` commands and options. Use this page to look up the syntax, default values, and effect of every command-line flag.

Synopsis

```
md-validate [--verbose] [--quiet] COMMAND [OPTIONS]


Commands:
  validate    Validate a Markdown file or directory
  repl        Start the interactive REPL
```

Global options

These options apply to all subcommands and must appear before the subcommand name.

Option	Default	Description
`--verbose`, `-v`	Off	Enable DEBUG-level logging. Use this when diagnosing unexpected results or rule failures.
`--quiet`, `-q`	Off	Suppress all non-error output. Only the exit code indicates pass or fail. Useful in CI pipelines that parse exit codes only.

validate — Validate a file or directory

```
md-validate validate TARGET --rules RULES [OPTIONS]
```

Option	Default	Description
`TARGET`	(required)	Path to a `.md` file or a directory. When a directory is provided, all `.md` files are validated recursively.
`--rules`, `-r`	(required)	Path to the JSON rule set file.

Option	Default	Description
`--output`, `-o`	None (print to stdout)	Directory where report files are written. One file is created per validated document. The directory is created if it does not exist.
`--format`, `-f`	`text`	Report format. One of: `text` (human-readable terminal output), `json` (machine-readable JSON object), `csv` (one row per rule).
`--workflows` / `--no-workflows`	Workflows enabled	Controls whether workflow chains are executed after individual rule evaluation. Use `--no-workflows` to evaluate rules only.

repl — Start the interactive REPL

```
md-validate repl
```

Starts the interactive REPL for rule development and debugging. No options are required. See the REPL commands reference *(on page 74)* for the list of REPL commands.

Examples

```
# Validate a single file with text output
md-validate validate article.md --rules rules/tutorial.json


# Validate a directory and save JSON reports
md-validate validate docs/ --rules rules/tutorial.json \
  --format json --output reports/


# Validate quietly (exit code only, no output)
md-validate --quiet validate docs/ --rules rules/tutorial.json


# Validate with debug logging
md-validate --verbose validate article.md --rules rules/tutorial.json
```

Exit codes reference

markdown-validator exits with code `0` when all Required rules pass and code `1` when any Required rule fails. CI systems use these codes to gate pull request merges.

Table 4. Exit codes

Code	Meaning	Effect in CI
`0`	All Required rules passed	CI step succeeds. The pipeline continues. Suggested-rule failures do not affect this code.
`1`	One or more Required rules failed	CI step fails. The pipeline stops (unless configured to continue on error). The merge is blocked.

Notes

- Suggested-level failures do not change the exit code. A file with only Suggested failures exits with code `0`.
- A parse error (file has no front matter) produces exit code `1`. The file is counted as a failed scan.
- In a directory scan, one failing file sets the exit code to `1` for the entire run.

Checking the exit code in a shell script

```
md-validate validate docs/ --rules rules/tutorial.json

if [ $? -eq 0 ]; then

  echo "All required rules passed."

else

  echo "Validation failed. Check the report for details."

fi
```

Validation report output reference

Reference for the structure and fields of markdown-validator validation reports in text, JSON, and CSV format.

Text format

Text format is the default. It is designed for reading in a terminal.

```
Validating: docs/tutorial-deploy-app.md

Rule set:   rules/tutorial.json
```

```
  ✓  [1]  topic-must-be-tutorial

  ✗  [4]  description-must-exist              [Required]

          → Add a description field to the front matter.

  ✗  [6]  title-under-60-chars                [Suggested]

          → Title must be under 60 characters.


Score: 4/6   FAILED
```

Table 5. Text report fields

Field	Description
✓ / ✗	Check mark = rule passed. Cross = rule failed.
`[N]`	Rule ID number.
Rule name	Human-readable name of the rule as defined in the rule set.
`[Required]` / `[Suggested]`	Severity level. Shown for failed rules. Required failures block the pipeline.
`→ message`	Mitigation message. Plain-English description of what to fix. Shown only for failed rules.
`Score: N/T`	N = rules passed, T = total rules evaluated.

JSON format

JSON format produces a single object with all validation results.

```
{
  "filepath": "docs/tutorial-deploy-app.md",
  "score": 4,
  "total_rules": 6,
  "passed": false,
  "results": [
    {
      "rule_id": 1,
      "rule_name": "topic-must-be-tutorial",
      "passed": true,
      "level": "Required",
      "expected_value": "tutorial",
      "actual_value": "tutorial",
```

```
      "mitigation": "",
      "filepath": "docs/tutorial-deploy-app.md"
    },
    {
      "rule_id": 4,
      "rule_name": "description-must-exist",
      "passed": false,
      "level": "Required",
      "expected_value": "true",
      "actual_value": "",
      "mitigation": "Add a description field to the front matter.",
      "filepath": "docs/tutorial-deploy-app.md"
    }
  ]
}
```

Table 6. JSON report fields

Field	Type	Description
`filepath`	string	Path to the validated document.
`score`	integer	Number of rules that passed.
`total_rules`	integer	Total number of rules evaluated.
`passed`	boolean	`true` if all Required rules passed.
`results[].rule_id`	integer	Rule identifier.
`results[].passed`	boolean	Whether this individual rule passed.
`results[].mitigation`	string	Remediation message. Empty string if the rule passed.

CSV format

CSV format produces one row per rule evaluation. Useful for importing into a spreadsheet or BI tool.

Table 7. CSV columns

Column	Description
`filepath`	Path to the validated document.
`rule_id`	Rule identifier.

Table 7. CSV columns (continued)

Column	Description
`rule_name`	Human-readable rule name.
`type`	`header` or `body`.
`level`	`Required` or `Suggested`.
`passed`	`true` or `false`.

Chapter 6. Understanding rule sets

Understanding rule sets

A rule set is a JSON file that tells markdown-validator what to check in a Markdown document. It contains two sections: `rules`, which defines individual checks, and `workflows`, which chains rules together with conditional logic.

Rule set structure

A rule set JSON file has the following top-level structure:

```
{
  "rules": {
    "header": [ ... ],
    "body":   [ ... ]
  },
  "workflows": [ ... ]
}
```

`rules.header`

An array of rules that check the YAML front-matter metadata at the top of each Markdown file. Use header rules to validate fields like `ms.topic`, `ms.date`, `title`, and `description`.

`rules.body`

An array of rules that check the document content — headings, section structure, text patterns, paragraph length. Body rules use XPath expressions to target specific elements in the rendered HTML.

`workflows`

An optional array of workflow definitions. Workflows chain multiple rules together with conditional logic ("if rule A passes, then also check rule B"). If you do not need conditional logic, this array can be empty or omitted.

Header rules vs. body rules

Table 8. Header rules vs. body rules

Aspect	Header rules	Body rules
What they check	YAML front-matter key-value pairs	Markdown content rendered as HTML

Table 8. Header rules vs. body rules (continued)

Aspect	Header rules	Body rules
Query syntax	Metadata key name (for example, `ms.topic`)	XPath expression (for example, `/html/body/h1`)
Typical use cases	Required fields exist, field values match allowed values, date is recent	Heading count, section order, required headings present, title starts with keyword

A minimal rule set example

This rule set checks two things: the `ms.topic` front-matter field must equal `tutorial`, and the document body must contain exactly one H1 heading.

```
{
  "rules": {
    "header": [
      {
        "id": 1,
        "name": "topic-must-be-tutorial",
        "type": "header",
        "query": "ms.topic",
        "flag": "value",
        "operation": "==",
        "value": "tutorial",
        "level": "Required",
        "mitigation": "Set ms.topic: tutorial in the front matter."
      }
    ],
    "body": [
      {
        "id": 2,
        "name": "must-have-exactly-one-h1",
        "type": "body",
        "query": "/html/body/h1",
        "flag": "count",
        "operation": "==",
        "value": "1",
```

```
        "level": "Required",
        "mitigation": "The document must have exactly one H1 heading."
      }
    ]
  },
  "workflows": []
}
```

Next steps

- To understand each field in a rule, see Anatomy of a validation rule *(on page 41)*.
- To create your first rule set file, see Create a rule set file *(on page 52)*.

Anatomy of a validation rule

Every markdown-validator rule is a JSON object with nine fields. Understanding what each field does is the foundation for writing rules that express your editorial standards accurately.

The nine rule fields

```
{
  "id":        1,
  "name":      "topic-must-be-tutorial",
  "type":      "header",
  "query":     "ms.topic",
  "flag":      "value",
  "operation": "==",
  "value":     "tutorial",
  "level":     "Required",
  "mitigation":"Set ms.topic: tutorial in the front matter."
}
```

`id`

A unique integer that identifies this rule. IDs are used in workflow definitions to refer to rules. Assign IDs sequentially within a rule set and do not reuse them.

`name`

A human-readable label shown in validation reports. Use lowercase words separated by hyphens (for example, `description-must-exist`). Choose a name that reads like an assertion so the failure is immediately clear.

`type`

Either `"header"` or `"body"`. Header rules check YAML front-matter; body rules check the rendered HTML. You can omit this field if the rule is in the correct section of the rule set file — it is injected automatically.

`query`

What to look up in the document.

- For **header** rules: the metadata key name (for example, `"ms.topic"`).
- For **body** rules: an XPath expression against the rendered HTML (for example, `"/html/body/h1"`).

`flag`

How to process the query result before comparing it with `value`. For header rules, common flags are `value`, `check`, `date`, and `pattern`. For body rules, common flags are `count`, `text`, and `dom`. See the Flags reference *(on page 69)* for the complete list.

`operation`

The comparison operator applied between the processed query result and `value`. Examples: `"=="` (equals), `"!="` (not equals), `">"` (greater than), `"[]"` (contains). See the Operators reference *(on page 71)* for the complete list.

`value`

The expected value the query result is compared against. Always a string. For numeric comparisons the string is coerced to a number at evaluation time. For multi-value checks (with `"[]"`), use a comma-separated list: `"guide, article, topic"`.

`level`

Severity of a failure: `"Required"` (blocks the CI pipeline) or `"Suggested"` (informational, does not block the pipeline). Defaults to `"Required"` if omitted.

`mitigation`

A plain-English message shown to authors when this rule fails. Write the mitigation as a direct instruction: what the author should do to fix the problem. This message is the primary communication between your rule and your authors — make it clear and actionable.

How the evaluation pipeline works

When markdown-validator evaluates a rule, it follows this sequence:

1. Look up the value of `query` in the document (either a metadata key or an XPath result).
2. Process the result with `flag` to extract a comparable string (for example, count the number of matching elements).
3. Apply `operation` to compare the processed result against `value`.
4. Return **pass** if the comparison is true, **fail** if it is false.

If the query returns nothing (key absent, no XPath matches), the rule fails with the `mitigation` message.

Understanding the workflow step language

Workflows chain rules together with conditional logic. Use them when a rule should only run if another rule passes or fails — for example, "check H2 structure only when the document is a tutorial." Simple checks that are always evaluated do not need workflows.

When to use workflows

Use a workflow when:

- A check is only meaningful if another check passes first. For example: the H1 must start with "Tutorial:" only applies when `ms.topic == tutorial`.
- You need to express conditional presence. For example: "if there are no H2 headings, then there should also be no H3 headings."
- Two checks together constitute a single editorial requirement. For example: either the Prerequisites heading exists, or the article is marked as a reference — but not both.

Do not use a workflow for checks that are always evaluated independently. Keep the rule list simple; add workflows only when the conditional logic is genuinely needed.

The workflow as a directed graph

Before reading the step token syntax, build this mental model: a workflow is a small directed graph. The nodes in the graph are:

`s` — Start

The fixed entry point. Every workflow begins here.

Rule IDs (1, 2, 3, …)

Each rule referenced in the workflow is a node. Execution passes through these nodes and each one produces a pass or fail result.

`D` — Decision

A branching node. The result of the rule just before `D` determines which branch runs next. A passing result takes the True (`T`) branch; a failing result takes the False (`F`) branch.

`M` — Merge

A convergence node. Both branches of a decision point lead back to `M` so the workflow has a single exit path.

`E` — End

The fixed exit point. Every workflow ends here.

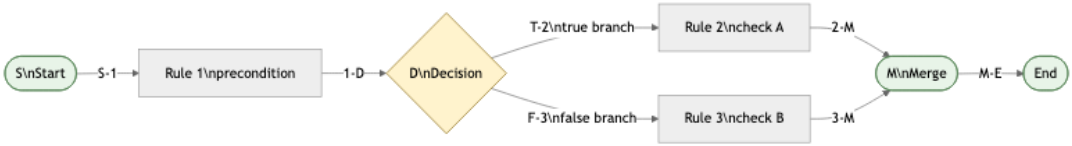
The `steps` field encodes the edges of this graph as a comma-separated list of `source-target` tokens. Each token is one directed edge: "execution can travel from *source* to *target*."

Reading step tokens

The conditional workflow token string and the graph it produces:

```
"steps": "S-1,1-D,T-2,F-3,2-M,3-M,M-E"
```

Figure 3. Graph of the conditional workflow S-1,1-D,T-2,F-3,2-M,3-M,M-E



Source: `docs-user/images/workflow-state-diagram.mmd`

Token-by-token reading:

Token	Meaning
`S-1`	Edge from Start to Rule 1. Rule 1 runs first.
`1-D`	Edge from Rule 1 to the Decision node. Rule 1's pass/fail result controls the branch.
`T-2`	True branch: if Rule 1 passed, run Rule 2.
`F-3`	False branch: if Rule 1 failed, run Rule 3.
`2-M`	Edge from Rule 2 to the Merge node.
`3-M`	Edge from Rule 3 to the Merge node.
`M-E`	Edge from Merge to End. The workflow is complete.

The simplest workflow: a single rule

The most common workflow is a linear, unconditional check:

```
"steps": "S-1,1-E"
```

Figure 4. Graph of the simple workflow S-1,1-E

Source: `docs-user/images/workflow-simple.mmd`

There is no decision node because the result is always the result of Rule 1. Wrapping a rule in a workflow (rather than including it in the plain rule list) allows you to assign a workflow-level `fix` message and a display name in the report.

Build a workflow from a plain-English requirement

The easiest way to write a workflow token string is to build it one edge at a time, starting from the requirement in plain English.

Requirement: "Tutorial articles must have an H1 that starts with 'Tutorial:'. Other article types are not checked for H1 format."

Step 1 — Identify the nodes. This requirement has a precondition (is this a tutorial?) and a conditional check (only for tutorials). That means the graph needs: S, a precondition rule, D, a check rule for the True branch, a sentinel rule for the False branch (see step 3), M, and E.

Assign IDs: Rule 1 checks `ms.topic == tutorial`. Rule 2 checks the H1 starts with "Tutorial:". Rule 10 is a sentinel (a rule that is known to always pass for any valid document).

Step 2 — Draw the edges one at a time.

Token added	Edge it encodes	Token string so far
`S-1`	Start → Rule 1 (precondition: is this a tutorial?)	`S-1`
`1-D`	Rule 1 → Decision (its result controls the branch)	`S-1,1-D`

Token added	Edge it encodes	Token string so far
`T-2`	True branch → Rule 2 (check H1 prefix for tutorials)	`S-1,1-D,T-2`
`F-10`	False branch → Rule 10 (sentinel; skip H1 check)	`S-1,1-D,T-2,F-10`
`2-M`	Rule 2 → Merge (True branch rejoins)	`S-1,1-D,T-2,F-10,2-M`
`10-M`	Rule 10 → Merge (False branch rejoins)	`S-1,1-D,T-2,F-10,2-M,10-M`
`M-E`	Merge → End (workflow complete)	`S-1,1-D,T-2,F-10,2-M,10-M,M-E`

Step 3 — Why the sentinel rule? The `F-M` token does not exist. Every branch must pass through a rule node before reaching M. When the False branch represents "nothing to check here," route it through a sentinel — a rule whose precondition always passes for well-formed documents. The sentinel contributes a pass result and the branch continues cleanly to M.

The finished workflow object:

```
{
  "name": "tutorial-h1-prefix",
  "steps": "S-1,1-D,T-2,F-10,2-M,10-M,M-E",
  "level": "Required",
  "fix": "Tutorial articles must open with an H1 that starts with 'Tutorial:'."
}
```

Practical example: enforcing different H1 formats per topic type

When both branches of a decision carry meaningful checks — not just "skip" — no sentinel is needed. This workflow enforces a different H1 prefix depending on `ms.topic`:

- If `ms.topic == tutorial` (Rule 1 passes), the H1 must start with "Tutorial:" (Rule 2).
- If `ms.topic != tutorial` (Rule 1 fails), the H1 must start with "How to" (Rule 3).

```
{
  "name": "h1-matches-topic-type",
  "steps": "S-1,1-D,T-2,F-3,2-M,3-M,M-E",
  "level": "Required",
```

```
  "fix": "Match your H1 prefix to your ms.topic value."
}
```

Rule 1 is the precondition; D routes to Rule 2 or Rule 3 depending on the topic type; both paths merge before the workflow ends. Because both branches run a real check, no sentinel is needed.

Avoid the workflow-inversion pattern

> **CAUTION:**
> The pattern described below is a known design smell. Do not use it for new rules. It exists only as a workaround until the `negate` flag is available in v0.3.

Some teams use workflow branching to express "this thing must NOT exist" — for example, "the title must not contain the word 'guide'." Because markdown-validator has no negation operator in v0.2, this requires inverting the pass/fail result by routing branches in reverse: the True branch (content IS present — the bad state) loops back into a failing rule, and the False branch (content is absent — the good state) routes to a sentinel that passes.

This pattern is fragile. A single miswiring of T and F branches causes the rule to silently pass bad content or fail good content — and the error is difficult to spot by reading the token string alone. If you must use it, test it exhaustively and document the inversion clearly in your rule set's README. For the full pattern and its known failure modes, see Pattern 8: Absence checks (design smell — avoid until v0.3) *(on page 50)*.

When v0.3 ships, replace any inverted workflows with a standard rule that uses `"negate": true`.

Next steps

- For the full step-pattern reference, see Workflow step patterns reference *(on page 72)*.
- To write a workflow, see Write a workflow for conditional rule chains *(on page 57)*.

Rule design patterns for common editorial checks

These patterns cover the most common editorial requirements you will need to express as rules. Each pattern includes the JSON, an explanation of how it works, and common variations.

Quick-reference: choose your pattern

Find the check you want to enforce in the table below, then jump to the corresponding pattern for the full rule JSON and explanation.

What you want to check	Rule type	Pattern to use
A front-matter field must exist (any value is acceptable)	header	Pattern 1: Require a front-matter field to exist *(on page 48)*
A front-matter field must equal a specific value, or one of a list	header	Pattern 2: Enforce an exact front-matter value *(on page 48)*
A date field must be within N days of today	header	Pattern 3: Check date freshness *(on page 49)*
The document must have exactly N headings of a given level	body	Pattern 4: Require exactly one H1 *(on page 49)*
A specific section heading (e.g., "Prerequisites") must exist	body	Pattern 5: Require a specific H2 section *(on page 49)*
An H1 (or other heading) must begin with a required prefix	body	Pattern 6: Enforce an H1 naming convention *(on page 50)*
A heading or field value must not exceed a character limit	body or header	Pattern 7: Limit title length *(on page 50)*
Something must NOT be present — avoid this in v0.2; use Pattern 8 only as a last resort	body or header	Pattern 8: Absence checks (design smell — avoid until v0.3) *(on page 50)*

Pattern 1: Require a front-matter field to exist

Use the `check` flag to verify a key is present, regardless of its value.

```
{
  "id": 1, "name": "description-must-exist", "type": "header",
  "query": "description",
  "flag": "check", "operation": "==", "value": "true",
  "level": "Required",
  "mitigation": "Add a description field to the front matter."
}
```

Pattern 2: Enforce an exact front-matter value

Use the `value` flag with `==` to require a specific value.

```
{
  "id": 2, "name": "topic-must-be-tutorial", "type": "header",
```

```
  "query": "ms.topic",
  "flag": "value", "operation": "==", "value": "tutorial",
  "level": "Required",
  "mitigation": "Set ms.topic: tutorial in the front matter."
}
```

To allow any of several values, use the `[]` operator with a comma-separated list:

```
"flag": "value", "operation": "[]",
"value": "tutorial, how-to, concept, reference"
```

Pattern 3: Check date freshness

Use the `date` flag with the `<` operator. The `value` is the maximum age in days.

```
{
  "id": 3, "name": "ms-date-must-be-within-one-year", "type": "header",
  "query": "ms.date",
  "flag": "date", "operation": "<", "value": "365",
  "level": "Suggested",
  "mitigation": "Update ms.date to today's date (YYYY-MM-DD format)."
}
```

The comparison is: `(today - ms.date).days < 365`.

Pattern 4: Require exactly one H1

Use the `count` flag on an XPath expression to count matching elements.

```
{
  "id": 4, "name": "must-have-exactly-one-h1", "type": "body",
  "query": "/html/body/h1",
  "flag": "count", "operation": "==", "value": "1",
  "level": "Required",
  "mitigation": "The document must have exactly one H1 heading."
}
```

Pattern 5: Require a specific H2 section

Use the `count` flag and XPath predicate to check that a specific section heading exists.

```
{
  "id": 5, "name": "must-have-prerequisites-section", "type": "body",
  "query": "//h2[normalize-space(text())='Prerequisites']",
  "flag": "count", "operation": ">", "value": "0",
  "level": "Required",
  "mitigation": "Add a '## Prerequisites' section to the document."
}
```

Pattern 6: Enforce an H1 naming convention

Use the `text` flag with `[:` (starts with) to check the content of the H1.

```
{
  "id": 6, "name": "h1-must-start-with-tutorial", "type": "body",
  "query": "/html/body/h1",
  "flag": "text", "operation": "[:", "value": "Tutorial:",
  "level": "Required",
  "mitigation": "H1 must begin with 'Tutorial:' for tutorial articles."
}
```

Pattern 7: Limit title length

Use the `l` operator (length limit) to ensure the H1 text is under a character threshold.

```
{
  "id": 7, "name": "h1-under-60-chars", "type": "body",
  "query": "/html/body/h1",
  "flag": "text", "operation": "l", "value": "60",
  "level": "Suggested",
  "mitigation": "H1 must be under 60 characters for SEO."
}
```

Pattern 8: Absence checks (design smell — avoid until v0.3)

> ⚠ **DANGER:**
> **Do not use this pattern for new rules.** markdown-validator v0.2 has no negation operator. Expressing "this must NOT exist" requires a workflow-inversion technique that is fragile and easy to wire incorrectly. A `negate` flag is planned for v0.3 and will replace this entirely. See Known

limitations *(on page 64)*. Only use the workaround below if you have an immediate need and cannot wait for v0.3.

To express "the title must NOT contain the word 'guide'," a workflow can invert a contains check using a known-passing sentinel rule. The inversion works by swapping which branch is "good": the True branch (word IS present — the bad state) is routed back to the failing rule, while the False branch (word is absent — the good state) routes to a sentinel that passes. **This is not standard workflow logic.** It exploits branching as a substitute for a missing operator.

```
{
  "name": "title-must-not-contain-guide",
  "steps": "S-5,5-D,F-1,T-5,5-M,M-E",
  "fix": "Remove the word 'guide' from the title."
}
```

Rule 5 checks `title contains "guide"` (passes when the word IS present — the bad state). Rule 1 is the sentinel. The F branch routes to the sentinel (good: word absent), and the T branch re-runs rule 5 (bad: word present, will fail again).

If you must use this pattern: test it exhaustively with both passing and failing documents, comment the inversion clearly in the rule set JSON, and add a migration note to replace it when v0.3 ships. See Avoid the workflow-inversion pattern *(on page 47)* for the full failure-mode analysis.

Chapter 7. Author rules

Create a rule set file

Create a new JSON rule set file that markdown-validator can use to validate your documentation. Start with a minimal structure and add rules incrementally.

markdown-validator is installed. See Install markdown-validator *(on page 18)*. Review the rule set structure before creating your file: Understanding rule sets *(on page 39)*.

A rule set file is a plain JSON file. Create it in the `rules/` directory of your documentation repository so that it is version-controlled alongside the content it governs.

1. Create a new file named after the content type it governs.

   Use a descriptive name that reflects the content type, for example `rules/tutorial.json` or `rules/reference.json`.

2. Add the top-level JSON structure.

```
{
  "rules": {
    "header": [],
    "body":   []
  },
  "workflows": []
}
```

   This is a valid, empty rule set. No rules means no checks — all files will pass until you add rules.

3. Add your first header rule to check a required front-matter field.

```
{
  "rules": {
    "header": [
      {
        "id": 1,
        "name": "ms-topic-must-exist",
        "type": "header",
        "query": "ms.topic",
        "flag": "check",
        "operation": "==",
```

```
        "value": "true",
        "level": "Required",
        "mitigation": "Add ms.topic to the front matter."
      }
    ],
    "body": []
  },
  "workflows": []
}
```

4. Test the rule set by running it against a sample document.

```
md-validate validate sample.md --rules rules/tutorial.json
```

   If the JSON is malformed, the tool reports a schema error. Fix the JSON and re-run.

   The validation report confirms the rule set loads and evaluates correctly.

You have a working rule set file with one rule. Add more rules incrementally, testing after each addition.

- To add header rules, see Write a header rule to check front-matter metadata *(on page 53)*.
- To add body rules, see Write a body rule to check document structure *(on page 55)*.

Write a header rule to check front-matter metadata

Header rules check the YAML front-matter metadata at the top of your Markdown files. Use them to enforce required fields, expected values, date freshness, and naming patterns.

You have an existing rule set file. See Create a rule set file *(on page 52)*. Understand the rule fields before proceeding: Anatomy of a validation rule *(on page 41)*.

1. Identify the front-matter field you want to check and the editorial requirement it must satisfy.

   Examples:

   - The `description` field must exist.
   - The `ms.topic` field must equal `tutorial`.
   - The `ms.date` field must be within the last 365 days.
2. Choose the flag that matches how you want to process the field value.

Table 9. Flag selection for header rules

You want to check...	Use flag
The field exists (any value is acceptable)	`check`
The field equals a specific value	`value`
The field's date is within N days	`date`
The field matches a regular-expression pattern	`pattern`

3. Choose the operator and expected value.
   - For `check`: use `"operation": "=="` and `"value": "true"`.
   - For exact string match: use `"operation": "=="` and the expected string.
   - For allowed-values set: use `"operation": "[]"` and a comma-separated list in `value`.
   - For date freshness: use `"operation": "<"` and the maximum age in days.
4. Write a clear, actionable mitigation message.

   The mitigation message is shown directly to authors when their document fails this rule. Write it as an imperative sentence that tells them exactly what to do. Compare:

   - **Unclear:** "ms.topic value is invalid."
   - **Clear:** "Set ms.topic to one of: tutorial, how-to, concept, reference."
5. Add the rule object to the `header` array in your rule set file.

```
{
  "id": 3,
  "name": "ms-topic-must-be-allowed-value",
  "type": "header",
  "query": "ms.topic",
  "flag": "value",
  "operation": "[]",
  "value": "tutorial, how-to, concept, reference",
  "level": "Required",
  "mitigation": "Set ms.topic to one of: tutorial, how-to, concept, reference."
}
```

6. Test the rule against a document that should pass and a document that should fail.

```
md-validate validate passing-doc.md --rules rules/tutorial.json

md-validate validate failing-doc.md --rules rules/tutorial.json
```

Confirm that the passing document scores correctly and the failing document shows your mitigation message. For interactive testing, see Test a rule in the interactive REPL *(on page 60)*.

The header rule is added to your rule set and validates correctly against sample documents.

Write a body rule to check document structure

Body rules check the content of your Markdown documents — headings, sections, paragraph structure, and text patterns. They use XPath expressions to query the rendered HTML. Test your XPath expressions in the REPL before adding them to a rule.

- You have an existing rule set file. See Create a rule set file *(on page 52)*.
- The interactive REPL is available for XPath testing. See Start the interactive REPL *(on page 59)*.

Body rules are more complex than header rules because XPath expressions can behave unexpectedly when documents have unusual structure. Always test your XPath in the REPL before committing a body rule. See the known limitations before writing queries that rely on within-section containment: Known limitations *(on page 64)*.

1. Identify the structural property you want to check.

   Common body checks:

   - The document has exactly one H1
   - A specific H2 section heading is present (for example, "Prerequisites")
   - The H1 starts with a required prefix
   - The H1 is under a character limit
   - The body contains at least one code block
2. Write an XPath expression that selects the element you want to check.

   Common XPath patterns for Markdown documents:

   Table 10. Common XPath patterns

What to select	XPath
The H1 element	`/html/body/h1`
All H2 elements	`//h2`

What to select	XPath
A specific H2 by text	`//h2[normalize-space(text())='Prerequisites']`
The second H2	`//h2[2]`
All code blocks	`//pre/code`
The entire body text	`/html/body`

3. Test your XPath expression in the REPL before adding it to the rule set.

```
md-validate repl

> load docs/sample-tutorial.md

> query //h2[normalize-space(text())='Prerequisites'] count
```

Confirm the query returns the expected result. See Test a rule in the interactive REPL *(on page 60)* for the full testing workflow.

4. Choose the flag based on what aspect of the XPath result you want to evaluate.

Table 11. Flag selection for body rules

You want to evaluate...	Use flag
How many elements matched	`count`
The text content of the first matched element	`text`
The HTML tag names of matched elements	`dom`
The full plain text of the entire body	`all`

5. Add the body rule object to the `body` array in your rule set file.

```
{
  "id": 5,
  "name": "must-have-prerequisites-section",
  "type": "body",
  "query": "//h2[normalize-space(text())='Prerequisites']",
  "flag": "count",
  "operation": ">",
  "value": "0",
  "level": "Required",
```

```
  "mitigation": "Add a '## Prerequisites' section to the document."
}
```

6. Validate sample documents to confirm the rule behaves correctly.

```
md-validate validate docs/sample-with-prereqs.md --rules rules/tutorial.json
md-validate validate docs/sample-without-prereqs.md --rules rules/tutorial.json
```

   The document with the section passes rule 5; the document without it fails with your mitigation message.

The body rule is added and validated. It correctly identifies documents that are missing the required structural element.

Write a workflow for conditional rule chains

A workflow chains rules together so that some checks only run when others pass or fail. Write a workflow when your editorial requirement is conditional — it only applies in specific circumstances.

- You have defined the individual rules the workflow will reference in your rule set file.
- Review the workflow concept: Understanding the workflow step language *(on page 43)*.
- Review the step-pattern reference: Workflow step patterns reference *(on page 72)*.

Before writing the workflow, clarify the conditional logic in plain English. For example: "If `ms.topic` is 'tutorial' (rule 1 passes), then also check that the H1 starts with 'Tutorial:' (rule 2). If it is not 'tutorial' (rule 1 fails), pass without checking rule 2."

1. Write out the conditional logic in plain English before translating it to the step language.

   Ask yourself:

   - What is the precondition rule? (the decision point)
   - What should happen when the precondition passes? (the True branch)
   - What should happen when the precondition fails? (the False branch — often "pass without further checks")
2. Map the plain-English logic to the step token sequence.

   For a simple conditional check (check rule B only when rule A passes):

```
S-A, A-D, T-B, F-sentinel, B-M, sentinel-M, M-E
```

   Where `sentinel` is the ID of a rule that always passes for valid documents (typically rule 1 in your rule set).

Example: check H1 format only when `ms.topic == tutorial` (rule 1) passes:

```
"steps": "S-1,1-D,T-6,F-1,6-M,1-M,M-E"
```

3. Add the workflow object to the `workflows` array in your rule set file.

```
{
  "name": "h1-format-for-tutorials",
  "steps": "S-1,1-D,T-6,F-1,6-M,1-M,M-E",
  "level": "Required",
  "fix": "For tutorial articles, H1 must start with 'Tutorial:'"
}
```

4. Test the workflow against documents that cover both branches.

```
# Document where ms.topic = tutorial (should evaluate rule 6)
md-validate validate tutorial-article.md --rules rules/tutorial.json


# Document where ms.topic != tutorial (should skip rule 6)
md-validate validate how-to-article.md --rules rules/tutorial.json
```

For interactive workflow testing, see Test a workflow in the interactive REPL *(on page 61)*.

The workflow is in your rule set. It evaluates rule 6 only when rule 1 passes, and skips it otherwise.

Chapter 8. Test rules

Start the interactive REPL

The interactive REPL lets you load a Markdown document and explore its structure, test XPath queries, and evaluate rules interactively. Use it when writing new rules to confirm your queries return what you expect before adding them to a rule set.

markdown-validator is installed and the virtual environment is active. See Install markdown-validator *(on page 18)*.

The REPL is a command-line tool for rule authors. It is not used for production validation — use `md-validate validate` for that. Think of the REPL as a scratch pad where you can try queries and rules against real documents without modifying your rule set file.

1. Start the REPL.

   ```
   md-validate repl
   ```

   The terminal shows a welcome message and the `>` prompt:

   ```
   Markdown Validator REPL

   Type 'help' for a list of commands, 'quit' to exit.

   >
   ```

2. Load a Markdown document to work with.

   ```
   > load docs/tutorial-deploy-app.md
   ```

   The REPL confirms the file is loaded and shows the front-matter keys:

   ```
   Loaded: /abs/path/docs/tutorial-deploy-app.md

     Metadata keys: ['title', 'ms.topic', 'ms.date', 'author', 'description']
   ```

3. **Optional:** Inspect the document's metadata and HTML structure before testing rules.

   ```
   > dump metadata

   > dump html
   ```

   `dump metadata` prints the YAML front matter as a JSON object. `dump html` prints the rendered HTML body, which shows the exact structure your XPath queries will run against.

The REPL is running and a document is loaded. You can now test XPath queries and evaluate rules. See Test a rule in the interactive REPL *(on page 60)*.

To exit the REPL, enter `quit` or press **Ctrl+D**.

Test a rule in the interactive REPL

Test a rule in the REPL by running the XPath query against a loaded document and then evaluating the full rule object. This confirms your rule will behave as expected before you commit it.

The REPL is running and a document is loaded. See Start the interactive REPL *(on page 59)*.

1. Test your XPath expression with the `query` command.

   Syntax: `query <xpath> <flag>`

   ```
   > query /html/body/h1 text

   Result: ['My Tutorial Title']


   > query //h2 count

   Result: [3]


   > query //h2 text

   Result: ['Prerequisites', 'Deploy the app', 'Clean up resources']
   ```

   Supported flags for `query`: `text`, `count`, `dom`.

   Confirm the result matches what you expect. If it does not, adjust the XPath and query again.

2. Evaluate the full rule JSON object with the `eval` command.

   Pass the rule as a single-line JSON object:

   ```
   > eval

    {"id":5,"name":"must-have-prerequisites","type":"body","query":"//

   h2[normalize-space(text())='Prerequisites']","flag":"count","operation":">","value":"0","mitigation":"Add a

    Prerequisites section."}
   ```

   The REPL prints PASS or FAIL with the rule name:

   ```
   PASS: rule id=5 name='must-have-prerequisites'
   ```

   or:

   ```
   FAIL: rule id=5 name='must-have-prerequisites'

     → Add a Prerequisites section.
   ```

3. Load a document that should fail the rule and confirm it does.

```
> load docs/article-without-prereqs.md

> eval {"id":5, ...same rule JSON...}
```

The REPL shows FAIL with your mitigation message, confirming the rule catches the intended problem.

Your rule is tested and behaves correctly on both passing and failing documents. Add it to the rule set file with confidence.

Test a workflow in the interactive REPL

Testing a workflow requires validating that both the True and False branches execute correctly. Use the full `md-validate validate` command with representative documents rather than the REPL `eval` command, because `eval` tests only a single rule.

Your workflow is written in the rule set file and you have at least two test documents: one that exercises the True branch and one that exercises the False branch.

The REPL's `eval` command evaluates a single rule, not a workflow chain. The best way to test a workflow is to run `md-validate validate` with `--verbose` to see detailed evaluation output.

1. Run validation on a document that should exercise the True branch of the workflow.

   ```
   md-validate --verbose validate docs/tutorial-article.md --rules rules/tutorial.json
   ```

   With `--verbose`, the output shows each rule evaluated and its result, including workflow steps.

   Confirm the workflow report shows the expected True-branch result for this document.
2. Run validation on a document that exercises the False branch.

   ```
   md-validate --verbose validate docs/how-to-article.md --rules rules/tutorial.json
   ```

   Confirm the workflow passes or fails as intended for this document type.
3. **Optional:** Use the REPL to inspect the metadata and structure of a document that produces unexpected results.

   ```
   md-validate repl

   > load docs/unexpected-result-doc.md

   > get ms.topic

   > query //h2 text
   ```

   Compare the actual metadata and structure against what your workflow rules expect. Common causes of unexpected results:

    - The precondition rule (the decision rule) matches more or fewer documents than you expected.
    - A sentinel rule (used in the False branch) is failing instead of always passing.
    - Whitespace in a metadata value causes an equality check to fail.

Both branches of the workflow evaluate correctly against representative documents.

Chapter 9. Govern and deploy

Rule set governance and versioning

A rule set is a living document. Good governance practices protect your authors from unexpected pipeline failures while allowing your editorial standards to evolve over time.

Store the rule set in version control

Commit your rule set file to the documentation repository alongside the content it governs. This ensures:

- Every author uses the same rule set version as CI.
- Changes to rules are tracked with the same history as content changes.
- Rollback is as simple as reverting the commit.

Recommended location: `rules/` at the root of the documentation repository.

Deciding between Required and Suggested

The level of a rule determines whether authors are blocked or merely informed. Use the following criteria:

Table 12. Level selection criteria

Situation	Level
The check enforces a hard contractual requirement (legal, accessibility, platform dependency)	Required
The check enforces a well-established style rule that all authors have been trained on	Required
You are introducing a new rule and want to monitor compliance before enforcing it	Suggested (promote to Required after 30–60 days)
The check is a recommendation that should be visible but should not block publication	Suggested

> **Tip:**
> Introduce new Required rules gradually. Start as Suggested, run reports to see how many articles fail, communicate the upcoming change to your authors, then promote to Required after the team has had time to address the backlog.

Communicating rule changes to authors

Authors who are blocked by a new rule they did not expect are frustrated. Follow this process when adding or promoting a rule:

1. Add the rule as `Suggested`.
2. Run a report across the repository to identify affected articles and their owners.
3. Notify authors of the upcoming change with enough lead time to fix their articles (at least two weeks for a large repository).
4. After the compliance window, change the level to `Required` and commit the update.

Deprecating and removing rules

When a rule no longer reflects your editorial standards:

1. Change the level from `Required` to `Suggested` to stop blocking authors while you plan the removal.
2. Remove the rule from the rule set file.
3. Update your rule set's changelog.

Do not change a rule's ID once it is in use. If you change the rule significantly, treat it as a new rule with a new ID and deprecate the old one.

Maintain a changelog

Keep a `CHANGELOG.md` alongside your rule set file to document what changed, when, and why. Authors who receive unexpected failures will search for this file first. At minimum, record:

- The date of each change
- Which rules were added, changed, or removed
- Whether a Suggested rule was promoted to Required
- The reason for the change

Known limitations

Understanding the current limitations of markdown-validator before you write rules saves hours of debugging. Four limitations affect the types of rules you can reliably express today.

Limitation 1: Flat HTML — within-section queries are not reliable

markdown-validator renders Markdown to HTML using the standard `markdown` library. This library produces **flat HTML**: all headings are direct children of `<body>`. There are no `<section>` wrappers that group a heading with its content.

This means XPath queries that rely on containment — "find a link that is inside the Prerequisites section" — will not work as intended. The query `.//h2[1]/following-sibling::a` does not return links within the first H2's section; it returns every link anywhere after the first H2 in document order.

What to do instead: Write rules that check element presence and order using absolute position (for example, `.//h2[1]` for the first H2), not containment. For sections, count the heading and rely on positional predicates.

Planned fix: A structured-DOM parser mode is planned for v0.4. See the Product Roadmap *(on page)*.

Limitation 2: No negation operator

There is no `negate` field or `not` operator. Expressing "the title must NOT contain the word 'guide'" requires a workflow-inversion workaround that is difficult to read and error-prone. See the absence-check pattern in Rule design patterns for common editorial checks *(on page 47)*.

Planned fix: A `negate: true` field on `RuleModel` is planned for v0.3.

Limitation 3: Multi-element equality checks are usually wrong

When an XPath query returns more than one element and you use `"operation": "=="`, the evaluator requires **every** element in the result set to equal the expected value. This is almost never the intended behavior.

Example of the problem: A rule targeting `".//h2[last()]/preceding-sibling::h2"` (to find the second-to-last H2) returns *all* H2s before the last one. Requiring all of them to equal "Clean up resources" fails on any document with more than two H2s.

What to do instead: Target the specific element you want by position, not by relative relationship. Use `".//h2[last()-1]"` to get the second-to-last H2 directly.

Limitation 4: NLTK language data must be installed separately

Rules that use the `s` (sentence count) or `p<N>` (part of speech) operators require NLTK language corpora that are not included with the pip package. A fresh environment will fail with a LookupError on first use of these rules.

What to do: Include the NLTK download step in your team's onboarding instructions and CI pipeline. See Download NLTK language data *(on page 19)*.

Deploy and version a rule set

Deploying a rule set means committing it to your documentation repository, verifying it does not create unexpected failures across existing content, and communicating any new requirements to your authors.

All new rules are tested. See Test a rule in the interactive REPL *(on page 60)* and Test a workflow in the interactive REPL *(on page 61)*. Review governance principles before deploying: Rule set governance and versioning *(on page 63)*.

1. Run the rule set across the entire documentation repository and save the results.

   ```
   md-validate validate docs/ --rules rules/tutorial.json \
     --format csv --output reports/pre-deploy-scan/
   ```

   Review the CSV reports to understand the blast radius of the new rules. How many articles fail? Which authors are affected?

2. If the number of failures is large, change new Required rules to Suggested before deploying.

   Deploying a Required rule that fails for 200 articles creates immediate pipeline failures for all authors. Start as Suggested, give authors time to comply, then promote to Required. See Rule set governance and versioning *(on page 63)*.

3. Update the rule set changelog.

   Document each change in `rules/CHANGELOG.md`:

   ```
   ## 2026-03-20


   ### Added

   - Rule 12: `description-under-160-chars` (Suggested) — description field

     must be under 160 characters for SEO.


   ### Changed
   ```

```
- Rule 3: `ms-date-must-be-fresh` promoted from Suggested to Required.

  Authors had 60 days to update stale dates.
```

4. Notify your author team of the changes before committing.

   Share the following information:

   - Which rules are new (with their mitigation messages)
   - Which Suggested rules are being promoted to Required, and when
   - The date the change takes effect
   - How to run the validation locally to check their articles
5. Commit the updated rule set and changelog.

```
git add rules/tutorial.json rules/CHANGELOG.md

git commit -m "rules: add description length check (Suggested)"

git push
```

   The CI pipeline uses the new rule set on all subsequent pull requests.

The rule set is deployed. Authors are informed of the changes and the CI pipeline enforces the updated rules.

Chapter 10. Reference

Rule fields reference

Complete reference for every field in a rule object. Use this page to look up allowed values, defaults, and field-level constraints.

Table 13. Rule object fields

Field	Type	Required	Default	Description
`id`	integer	Yes	—	Unique positive integer. Strings are coerced to integers at load time for backward compatibility. Do not reuse IDs within a rule set.
`name`	string	Yes	—	Human-readable label shown in reports. Recommended format: lowercase words separated by hyphens (`must-have-h1`).
`type`	string	Yes*	Injected from section	Either `"header"` or `"body"`. Can be omitted if the rule is in the correct `rules.header` or `rules.body` array — the tool injects the value automatically.
`query`	string	Yes	—	For header rules: the front-matter key name. For body rules: an XPath 1.0 expression against the rendered HTML.
`flag`	string	Yes	—	Processing mode. Controls how the query result is extracted before comparison. See Flags reference *(on page 69)*.
`operation`	string	Yes	—	Comparison operator token. See Operators reference *(on page 71)*.
`value`	string	Yes	—	Expected value for the comparison. Always a string; numeric coercion is applied by the operator. For multi-value checks, use a comma-separated list.
`level`	string	No	`"Required"`	Either `"Required"` or `"Suggested"`. Required failures block the CI pipeline (exit code 1). Suggest-

Table 13. Rule object fields (continued)

Field	Type	Required	Default	Description
				ed failures are reported but do not change the exit code.
`mitigation`	string	No	`""`	Plain-English remediation message shown to authors when the rule fails. Write as an imperative sentence: what to do to fix the problem.

Minimal valid rule

```
{
  "id": 1,
  "name": "topic-must-exist",
  "type": "header",
  "query": "ms.topic",
  "flag": "check",
  "operation": "==",
  "value": "true"
}
```

Flags reference

The `flag` field controls how a rule's query result is processed before comparison. Header and body rules each have their own flag set.

Header flags

Flag	Behavior	Example rule

`value`	Retrieves the raw string value of the front-matter key. Applies the operator and value comparison directly to this string.	`"query": "ms.topic",`
		`"flag": "value",`
		`"operation": "==",`
		`"value": "tutorial"`

`check`	Returns `true` if the key exists in the front matter, `false` if absent. The `operation` and `value` fields are ignored.	`"query": "description",` `"flag": "check",`

Flag	Behavior	Example rule
	Use `"operation": "=="` and `"value": "true"` by convention.	`"operation": "==",` `"value": "true"`

`date`	Parses the front-matter value as a date. Computes the age in days as `(today - date).days`. Applies the operator against `value` (the age threshold in days). Use `"operation": "<"` to require the date is fresher than N days.	`"query": "ms.date",`
		`"flag": "date",`
		`"operation": "<",`
		`"value": "365"`

`pattern`	Applies the operator as a regular expression match against the front-matter value. Equivalent to using the `r` operator on a `value` flag.	`"query": "author",`
		`"flag": "pattern",`
		`"operation": "r",`
		`"value": "^[a-z0-9-]+$"`

Body flags

Flag	Behavior	Example rule

`count`	Counts the number of XPath matches. Returns the count as a string for comparison.	`"query": "/html/body/h1",`
		`"flag": "count",`
		`"operation": "==",`
		`"value": "1"`

`text`	Extracts the text content of the first XPath match. Returns an empty string if there are no matches.	`"query": "/html/body/h1",`
		`"flag": "text",`
		`"operation": "l",`
		`"value": "60"`

`dom`	Returns a comma-separated list of HTML tag names of the matched elements. Useful for checking element type.	`"query": "/html/body/*[2]",`
		`"flag": "dom",`
		`"operation": "==",`
		`"value": "h2"`

`all`	Extracts all text content of the matched elements, including nested	`"query": "/html/body",` `"flag": "all",`

Flag	Behavior	Example rule
	text. Typically used with `/html/body` to operate on the full page text.	`"operation": "s",` `"value": "500"`

Operators reference

The `operation` field specifies the comparison operator applied between the processed query result and the expected `value`. The operator returns true (pass) or false (fail).

Table 14. Comparison operators

Token	Name	Behavior	Example
`==`	Equals	True if the result equals the value after whitespace stripping. Case-sensitive.	`"operation": "==", "value": "tutorial"`
`!=`	Not equals	True if the result does not equal the value after whitespace stripping.	`"operation": "!=", "value": "draft"`
`>`	Greater than	Numeric comparison. Both operands coerced to integers.	`"operation": ">", "value": "0"`
`<`	Less than	Numeric comparison. Both operands coerced to integers.	`"operation": "<", "value": "365"`
`[]`	Contains	True if the result contains the value as a substring. Case-insensitive. For comma-separated value lists, the result must contain at least one of the listed values.	`"operation": "[]", "value": "tutorial, how-to"`
`[:`	Starts with	True if the result starts with the value string.	`"operation": "[:", "value": "Tutorial:"`
`:]`	Ends with	True if the result ends with the value string.	`"operation": ":]", "value": "resources"`
`r`	Regex match	True if Python `re.search(value, result)` matches. Uses DOTALL mode (dot matches newlines).	`"operation": "r", "value": "^Tutorial: [A-Z]"`

Table 14. Comparison operators (continued)

Token	Name	Behavior	Example
`l`	Length limit	True if `len(result) < int(value)`. Use to enforce maximum character counts.	`"operation": "l", "value": "60"`
`s`	Max sentences	True if `sentence_count <= int(value)`. Uses NLTK tokenizer. Requires NLTK corpora.	`"operation": "s", "value": "3"`
`p<N>`	Part of speech	True if the Nth word's Penn Treebank POS tag equals value. Position is 1-indexed. Requires NLTK corpora. Common tags: `NN` (noun), `VB` (base verb), `JJ` (adjective).	`"operation": "p3", "value": "VB"`

Notes

- The `s` and `p<N>` operators require NLTK corpora to be installed. See Download NLTK language data *(on page 19)*.
- There is currently no negation operator. See the absence-check pattern in Rule design patterns for common editorial checks *(on page 47)*.
- For multi-element XPath results with `==`, the evaluator requires every element to match. Target specific elements by position to avoid this. See Known limitations *(on page 64)*.

Workflow step patterns reference

A workflow's `steps` field is a comma-separated string of `source-target` tokens. Twelve patterns are recognised. Combine them to express any conditional rule dependency.

Table 15. Workflow step patterns

Pattern	Meaning	Used in context
`S-N`	Start: load rule N as the initial state.	`S-1` — start with rule 1
`N-D`	Rule N result becomes the decision point (the branch condition).	`1-D` — rule 1 is the decision
`M-D`	Merge state becomes the decision point.	Used in loops or multi-rule merges

Table 15. Workflow step patterns (continued)

Pattern	Meaning	Used in context
`T-N`	If the current decision is True (pass), load rule N.	`T-2` — if True, run rule 2
`F-N`	If the current decision is False (fail), load rule N.	`F-3` — if False, run rule 3
`T-R`	If the current decision is True, reverse (negate) it.	Rarely used; present to enable inversion patterns
`F-R`	If the current decision is False, reverse (negate) it.	Used in absence-check workarounds
`N-M`	Rule N result exits into the merge state.	`2-M` — rule 2 feeds into merge
`M-N`	Merge state exits to rule N.	Continues workflow after a merge
`M-E`	Merge state ends the workflow.	`M-E` — terminate at merge
`N-E`	Rule N result ends the workflow.	`1-E` — terminate with rule 1's result
`N-N`	Both rules must pass (logical AND). The second N is the rule that follows the first.	`1-2` — rules 1 AND 2 must both pass

Recipe: Single rule check

```
"steps": "S-1,1-E"
```

Start with rule 1. Rule 1's result ends the workflow.

Recipe: Conditional check (run rule B only when rule A passes)

```
"steps": "S-A,A-D,T-B,F-sentinel,B-M,sentinel-M,M-E"
```

Rule A is the decision. If True, rule B runs and its result feeds into merge. If False, the sentinel rule (a known-passing rule) feeds into merge. Merge ends the workflow.

Recipe: Both rules must pass (AND)

```
"steps": "S-1,1-2,2-E"
```

Start with rule 1. Rule 1 must pass before rule 2 is checked. Rule 2's result ends the workflow.

Notes

- Both `S-1,1-E` and `(S,1)(1,E)` formats are accepted; the parenthesis format is normalised at load time.
- Rule IDs referenced in step tokens must exist in the `rules` section of the same rule set file.

REPL commands reference

Complete reference for all commands available in the `md-validate repl` interactive session.

Table 16. REPL commands

Command	Syntax	Description
`load`	`load <path>`	Load a Markdown file. All subsequent commands operate on this file. You can load a new file at any time to switch context.
`dump metadata`	`dump metadata`	Print the loaded file's YAML front matter as a formatted JSON object. Use this to confirm which keys are present and their exact values before writing header rules.
`dump html`	`dump html`	Print the rendered HTML of the loaded file's body. Use this to see the exact DOM structure that body-rule XPath expressions will query.
`query`	`query <xpath> [flag]`	Run an XPath expression against the loaded file's HTML body. Optional flag: `text` (default), `count`, or `dom`. Returns the result as a list.
`get`	`get <key>`	Retrieve the value of a front-matter key. Equivalent to `dump metadata` for a single key.
`eval`	`eval <json-rule>`	Evaluate a rule JSON object against the loaded document. The rule must be valid JSON on a single line. Returns PASS or FAIL with the rule name and, on failure, the mitigation message.
`help`	`help`	List all available commands.
`quit`	`quit` or `exit`	Exit the REPL.

Example session

```
> load docs/tutorial-deploy-app.md

Loaded: /abs/path/docs/tutorial-deploy-app.md

  Metadata keys: ['title', 'ms.topic', 'ms.date', 'author', 'description']



> get ms.topic

ms.topic: tutorial



> query /html/body/h1 text

Result: ['Tutorial: Deploy a web application']



> query //h2 count

Result: [4]



> query //h2 text

Result: ['Prerequisites', 'Deploy the app', 'Test the deployment',

         'Clean up resources']



> eval

 {"id":1,"name":"h1-must-start-with-tutorial","type":"body","query":"/html/body/h1","flag":"text","operation":"[:","val

ue":"Tutorial:","mitigation":"H1 must start with 'Tutorial:'"}

PASS: rule id=1 name='h1-must-start-with-tutorial'



> quit
```

Chapter 11. Troubleshooting

Resolve a NLTK LookupError on first run

A `LookupError: Resource punkt_tab not found` error means the NLTK language data required by sentence-count or part-of-speech rules is not installed on your system.

Situation

You run `md-validate validate` and see an error similar to:

```
LookupError: Resource punkt_tab not found.

Please use the NLTK Downloader to obtain the resource:

  >>> import nltk; nltk.download('punkt_tab')
```

This error occurs when a rule uses the `s` (sentence count) or `p<N>` (part of speech) operators and the required NLTK corpora are not installed.

Cause

The NLTK language data files (`punkt_tab` and `averaged_perceptron_tagger_eng`) are not included in the markdown-validator pip package. They must be downloaded once per Python environment before sentence or POS rules can run.

Resolution

1. Activate the virtual environment where markdown-validator is installed.

   ```
   source .venv/bin/activate       # macOS/Linux

   .venv\Scripts\Activate.ps1      # Windows PowerShell
   ```

2. Download the required NLTK corpora.

   ```
   python -m nltk.downloader punkt_tab averaged_perceptron_tagger_eng
   ```

   NLTK downloads the files to your NLTK data directory (`~/nltk_data` on macOS/Linux).

3. Re-run the validation.

   ```
   md-validate validate docs/article.md --rules rules/tutorial.json
   ```

   The LookupError no longer appears.

Cause (CI environment)

The CI pipeline does not include an NLTK download step.

Resolution (CI environment)

Add the NLTK download step to your CI workflow before the validation step. See Add markdown-validator to a GitHub Actions workflow *(on page 29)* for a complete GitHub Actions example that includes this step.

Resolve a ParseError caused by missing front matter

A `ParseError` means markdown-validator could not find a valid YAML front-matter block at the top of your file. Every Markdown file must begin with `---` to be validated.

Situation

You run validation and see an error similar to:

```
ParseError: docs/article.md — no front matter found.

  File must begin with a --- delimited YAML block.
```

The file is not validated and counts as a failure in the report.

Cause: Front matter block is absent

The file does not start with `---`, or the opening delimiter is preceded by whitespace or a blank line.

Resolution

1. Open the file in your editor and check the first line.
2. Ensure the very first line of the file is exactly `---` with no preceding content.

   A valid front matter block looks like this:

   ```
   ---
   title: "Deploy a web application"
   ms.topic: tutorial
   ms.date: 2026-03-20
   author: your-github-username
   description: "Step-by-step guide to deploying a web app."
   ---


   # Tutorial: Deploy a web application
   ...
   ```

3. Close the front matter block with a second `---` line.
4. Re-run validation to confirm the parse error is resolved.

Cause: File encoding issue

The file contains a byte-order mark (BOM) or non-UTF-8 characters before the `---` delimiter.

Resolution

Save the file as UTF-8 without BOM. In most text editors, this is the default encoding. If you are using VS Code, check the encoding indicator in the status bar at the bottom right — it should show **UTF-8**.

Debug a rule that never passes on a valid document

A rule that always fails even on correctly formatted documents usually has a mismatch between the query result and the expected value. Use the REPL to inspect the exact values the rule is working with.

Situation

You have a rule that you believe should pass on a correctly formatted document, but it always reports FAIL regardless of the document content.

Cause: Whitespace in the query result

The metadata value or XPath text result has leading or trailing whitespace that prevents an equality match.

Resolution

1. Start the REPL, load the document, and inspect the exact value.

   ```
   md-validate repl

   > load docs/article.md

   > get ms.topic
   ```

   Compare the output to what your rule's `value` field expects. Look for extra spaces or quotes.
2. Update the rule's `value` field to match the exact string, or clean up the front matter.

Cause: Multi-element XPath result with equality operator

An XPath expression returns multiple elements, and the `==` operator requires *every* element to match the expected value. If any element differs, the rule fails. This is almost never the intended behavior.

Resolution

1. In the REPL, run the query with the `text` flag to see all results.

```
> query //h2 text

Result: ['Prerequisites', 'Deploy the app', 'Clean up resources']
```

2. Narrow the XPath to target the specific element you want by position.

```
# Instead of: //h2[last()]/preceding-sibling::h2

# Use:

> query //h2[last()-1] text

Result: ['Deploy the app']
```

Cause: Wrong flag for the check type

Using `text` when you need `count`, or vice versa, causes the comparison to fail because the result is not in the expected form.

Resolution

1. Review the flags reference to confirm you are using the correct flag for your check.

   See Flags reference *(on page 69)* for flag descriptions and examples.

2. In the REPL, compare the output of different flags on the same XPath to see which form matches your expected value.

```
> query /html/body/h1 count

Result: 1


> query /html/body/h1 text

Result: ['How to deploy the app']
```

   Use `count` when comparing to a number, and `text` when comparing to a string.

Cause: Inverted workflow branch logic

A workflow intended to express absence uses a branch pattern that is inverted — it passes when the content is wrong and fails when the content is correct.

Resolution

1. Run validation with `--verbose` to trace the workflow step-by-step.

   ```
   md-validate --verbose validate docs/article.md --rules rules/tutorial.json
   ```

   The verbose output shows which branch (True or False) the decision node follows for each workflow step.

2. Read the decision point output carefully and verify your branch routing.

   The True branch runs when the precondition rule *passes*. If your workflow uses `F-sentinel` for the False branch, confirm the sentinel rule passes on valid documents by testing it in the REPL with `eval`.

   See Rule design patterns for common editorial checks *(on page 47)* for the absence-check pattern and the known fragility of workflow inversion.

Debug an XPath expression that returns unexpected results

XPath queries that seem correct but return unexpected results are usually caused by the flat HTML structure that markdown-validator produces. Use the REPL to inspect the actual HTML and adjust your query.

Situation

An XPath expression in a body rule returns no results, or it returns more elements than expected, causing the rule to behave incorrectly.

Cause: Flat HTML — within-section containment does not work

The rendered HTML is flat: all headings are direct children of `<body>`. Queries like `.//h2[1]/following-sibling::p` do not return only paragraphs in the first H2's section — they return all paragraphs after the first H2, including those in every subsequent section.

Resolution

1. In the REPL, run `dump html` on your document to see the actual HTML structure.

   ```
   md-validate repl

   > load docs/tutorial.md

   > dump html
   ```

   Confirm that headings are direct children of `<body>` with no section wrappers.

2. Rewrite the XPath to use absolute position instead of containment.

   Instead of:

   ```
   .//h2[@id='prerequisites']/following-sibling::p[1]
   ```

   Use:

   ```
   //h2[normalize-space(text())='Prerequisites']
   ```

   Checking for the presence of the heading rather than its content achieves the same editorial goal without requiring containment.

Cause: Whitespace in element text

XPath text comparisons are exact. A heading with a trailing space (`Prerequisites `) does not match the predicate `[text()='Prerequisites']`.

Resolution

1. In the REPL, run the query with `text` to see the exact string the element contains, including any whitespace.

   ```
   > query //h2 text

   Result: ['Prerequisites ', 'Deploy the app']
   ```

   Note any trailing spaces in the output.

2. Update the XPath predicate to use `normalize-space()` instead of a bare `text()` comparison.

   ```
   //h2[normalize-space(text())='Prerequisites']
   ```

Cause: XPath returns zero results

The element path does not match any nodes in the HTML. Common reasons: the element name is wrong, the document uses a heading level you did not expect, or the XPath predicate is too strict.

Resolution

1. In the REPL, run a broader query first and narrow it down.

   ```
   > query //h2 text

   Result: ['Prerequisites', 'Deploy the app', 'Clean up resources']
   ```

```
> query //h2[2] text

Result: ['Deploy the app']
```

2. Compare the actual element text against your predicate value.

Resolve a validation failure in a CI/CD pipeline

When the validation step in your CI pipeline fails, your pull request is blocked. This topic shows you how to find the failure report, reproduce the failure locally, and resolve the specific issue.

Situation

Your pull request shows a failing check in CI. The check is named something like **Validate documentation** or **validate**. The merge button is blocked.

Cause: A Required rule failed

One or more Required validation rules failed on at least one file in your pull request.

Resolution

1. Open the CI pipeline log to find the rule failure details.

   In GitHub Actions: navigate to the **Actions** tab of the repository, select the failing workflow run, expand the **Validate documentation** step, and read the output.

   Alternatively, if reports were uploaded as artifacts, download the **validation-reports** artifact and open the JSON file for your changed article.

2. Note the rule name and mitigation message for each failure.
3. Reproduce the failure locally.

   ```
   md-validate validate docs/your-article.md --rules rules/tutorial.json
   ```

   Confirm you see the same failure. If local validation passes but CI fails, check that your local rule set and the one in CI are the same version.

4. Fix the identified failures using the mitigation messages as your guide.

   See Read and act on a validation report *(on page 22)* for guidance on common failure types.

5. Push the fix and verify the CI check passes.

Cause: NLTK data is missing from the CI environment

The pipeline does not include a step to download NLTK corpora, or the NLTK cache step failed.

Resolution

1. Open your workflow file (`.github/workflows/validate-docs.yml`) and add the NLTK download step before the validation step.

```
- name: Download NLTK language data

  run: python -m nltk.downloader punkt_tab averaged_perceptron_tagger_eng
```

2. Commit the updated workflow file and push.
   The CI pipeline downloads the NLTK corpora before validation runs. For a complete workflow example including the NLTK cache step, see Add markdown-validator to a GitHub Actions workflow *(on page 29)*.

Cause: CI uses a different rule set than local validation

CI validates with the rule set file committed to the repository. If you are running a different version of the rule set locally, the results will differ.

Resolution

1. Check the CI configuration file to see which rule set it uses.

   Open `.github/workflows/validate-docs.yml` and look for the `--rules` flag in the validate step.

2. Run local validation with the same rule set path.

```
md-validate validate docs/your-article.md --rules rules/tutorial.json
```

Chapter 12. Reference

Glossary

body rule

A validation rule that checks the content of the Markdown document below the front-matter block. Body rules use XPath expressions to query the rendered HTML. Contrast with *header rule*.

CI/CD

Continuous Integration / Continuous Delivery. An automated pipeline that builds, tests, and validates code and content on every pull request. markdown-validator runs as a step in a CI/CD pipeline to gate merges on validation results.

exit code

An integer returned by a command-line program when it finishes. markdown-validator exits with `0` when all Required rules pass and `1` when any Required rule fails. CI systems use exit codes to determine whether a step succeeded or failed.

flag

A field in a validation rule that controls how the query result is processed before being compared to the expected value. Header flags include `value`, `check`, `date`, and `pattern`. Body flags include `count`, `text`, `dom`, and `all`. See also: *operator*.

front matter

A YAML metadata block at the top of a Markdown file, delimited by `---` lines. Front matter contains structured metadata such as `title`, `ms.topic`, `ms.date`, and `description`. markdown-validator requires every file to have a front-matter block.

header rule

A validation rule that checks the YAML front-matter metadata at the top of a Markdown file. Header rules use a metadata key name as their `query` field. Contrast with *body rule*.

mitigation message

A plain-English remediation message shown to an author when a rule fails. Defined in the rule's `mitigation` field. A good mitigation message is an imperative sentence that tells the author exactly what to change.

operator

A comparison function applied between the processed query result and the expected value in a rule. Operators include equality (`==`), inequality (`!=`), contains (`[]`), starts-with (`[:`), regex match (`r`), and length limit (`l`). See also: *flag*.

ParseError

An error raised when markdown-validator cannot read a `.md` file because it does not begin with a valid YAML front-matter block. A file that produces a ParseError is counted as a failed scan and contributes to the exit code of `1`.

REPL

Read–Eval–Print Loop. An interactive command-line session started with `md-validate repl`. The REPL lets rule creators load a Markdown file, inspect its structure, test XPath expressions, and evaluate rule JSON objects before committing rules to a rule set file.

rule

A JSON object that defines a single assertion about a Markdown document. A rule specifies what to query (`query`), how to process the result (`flag`), how to compare it (`operation`), and what the expected value is (`value`). Rules have a severity level (`Required` or `Suggested`) and a mitigation message.

rule set

A JSON file that defines all the validation rules and workflows for a specific content type. A rule set has three top-level elements: `rules.header`, `rules.body`, and `workflows`. Rule set files are version-controlled in the documentation repository and used by both local validation and CI.

scan report

The output produced by validating one Markdown file against a rule set. A scan report includes a score (passing rules / total rules), a per-rule pass/fail result, and mitigation messages for each failure. Available in text, JSON, and CSV formats.

score

The number of rules that passed divided by the total number of rules evaluated, expressed as a fraction (for example, `24/26`). A score of `26/26` means every rule passed. The score does not directly determine the CI exit code — only the pass/fail status of Required rules does.

Suggested

A rule severity level that indicates a best-practice recommendation. Suggested-level failures are reported but do not affect the CLI exit code. A CI pipeline step passes even if Suggested rules fail. Contrast with *Required*.

Required

A rule severity level that indicates a mandatory editorial standard. Required-level failures set the CLI exit code to `1`, causing a CI pipeline step to fail and blocking the merge. The default level if `level` is omitted from a rule. Contrast with *Suggested*.

workflow

A named sequence of rule evaluation steps that uses conditional branching. Workflows allow a check to run only when a precondition is met. Defined in the `workflows` array of a rule set file using the workflow step language.

XPath

XML Path Language. A query language for selecting nodes in an XML or HTML document. markdown-validator body rules use XPath 1.0 expressions to query the rendered HTML of a Markdown file's body. The HTML is flat: all headings are direct children of `<body>`.